



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Bitácora S08

Saga orquestada y circuit breaker en Mexi Banco

Asignatura: Sistemas Distribuidos
Profesor: Oscar Manuel Ruiz Hurtado
Grupo: 02
Semestre: 2027-1
Integrante: Arroyo Onofre Leonardo

Objetivo

Comprobar qué sucede cuando una transferencia se interrumpe entre el cargo y el abono, y corregir ese caso con una saga orquestada. Además, implementar un circuit breaker manual para controlar las llamadas al servicio de cuenta y observar sus estados cerrado, abierto y semiabierto.

Preparación del entorno

Antes de cualquier cosa, se cambió el timeout del gateway a 60 segundos. Esto fue necesario porque el ejercicio incluye una pausa de 20 segundos entre el cargo y el abono; si el gateway cortaba antes la espera, no se podía apreciar correctamente el caso. Para las pruebas se utilizó Postman y para levantar, detener y consultar los servicios se utilizó Docker Compose.

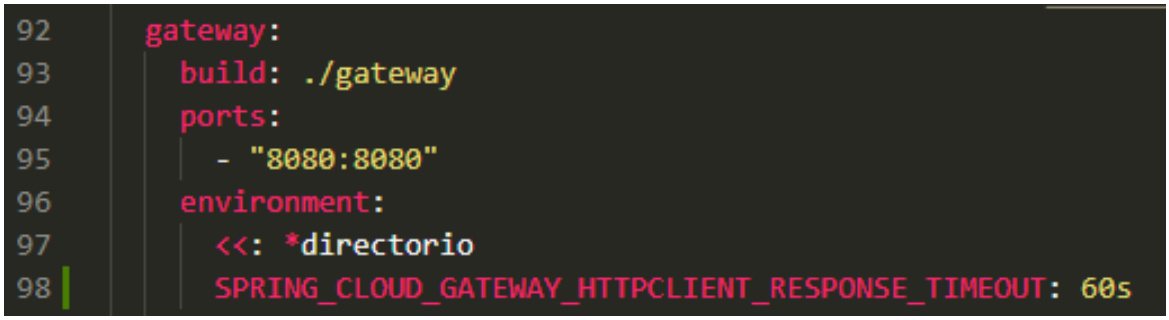


Figura 1: Timeout del gateway configurado en 60 segundos dentro de docker-compose.yml.

Nuevamente apareció el problema de los saltos de línea CRLF en mvnw. Para corregirlo se normalizó el archivo y se dio permiso de ejecución antes de invocar Maven. La línea utilizada fue RUN sed -i 's/\r$//' mvnw && chmod +x mvnw. Con esto se pudo continuar con la construcción de las imágenes.



Figura 2: Construcción y levantamiento de los servicios con Docker Compose.

```
PS C:\Users\arroy\Documents\9no semestre\SD\sd-2027-1\entregas\arroyo_leonardo\s08\mexi-banco-s08> docker compose ps
NAME                              IMAGE                             COMMAND                  SERVICE         CREATED          STATUS                    PORTS
mexi-banco-v06-cuenta-1           mexi-banco-v06-cuenta             "java -jar app.jar"      cuenta          49 seconds ago   Up 38 seconds (healthy)   8080/tcp
mexi-banco-v06-db-1               postgres:17-alpine                "docker-entrypoint.s…"   db              49 seconds ago   Up 49 seconds (healthy)   5432/tcp
mexi-banco-v06-discovery-1        mexi-banco-v06-discovery          "java -jar app.jar"      discovery       49 seconds ago   Up 49 seconds (healthy)   0.0.0.0:8761->8761/tcp, [::]:8761->876
1/tcp
mexi-banco-v06-gateway-1          mexi-banco-v06-gateway            "java -jar app.jar"      gateway         49 seconds ago   Up 38 seconds (healthy)   0.0.0.0:8080->8080/tcp, [::]:8080->808
0/tcp
mexi-banco-v06-movimiento-1       mexi-banco-v06-movimiento         "java -jar app.jar"      movimiento      49 seconds ago   Up 38 seconds (healthy)   8080/tcp
mexi-banco-v06-notificacion-1     mexi-banco-v06-notificacion       "java -jar app.jar"      notificacion    49 seconds ago   Up 38 seconds (healthy)   8080/tcp
mexi-banco-v06-spei-1             mexi-banco-v06-spei               "java -jar app.jar"      spei            49 seconds ago   Up 38 seconds (healthy)   8080/tcp
mexi-banco-v06-transferencia-1    mexi-banco-v06-transferencia      "java -jar app.jar"      transferencia   49 seconds ago   Up 38 seconds (healthy)   8080/tcp
```

Figura 3: Consulta del estado de los contenedores y sus healthchecks.

Para agilizar esta sesión se preparó una colección en Postman con cinco carpetas: problema original, saga exitosa, compensación exitosa, compensación pendiente y circuit breaker manual. Cada escenario utiliza un par distinto de cuentas, de modo que los saldos de una prueba no alteran los de la siguiente.

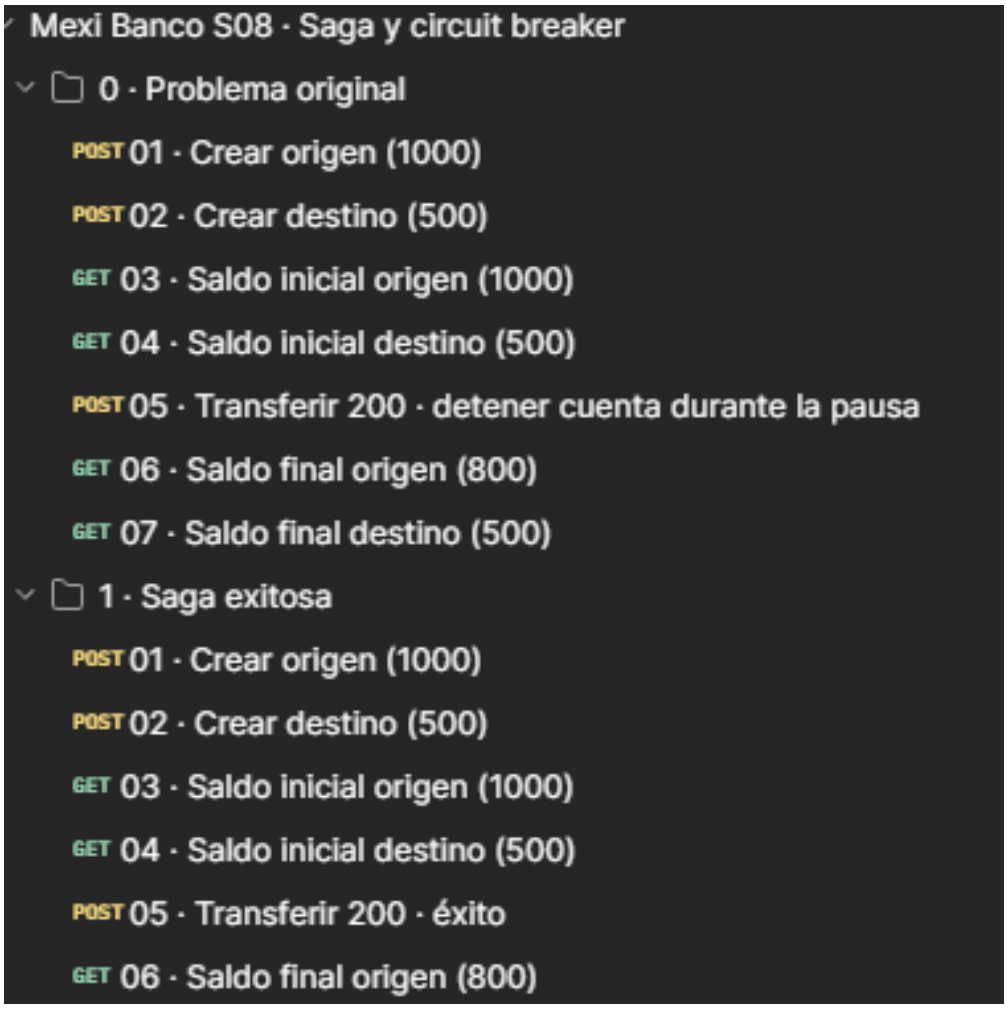


Figura 4: Organización de las peticiones de la colección Mexi Banco S08.

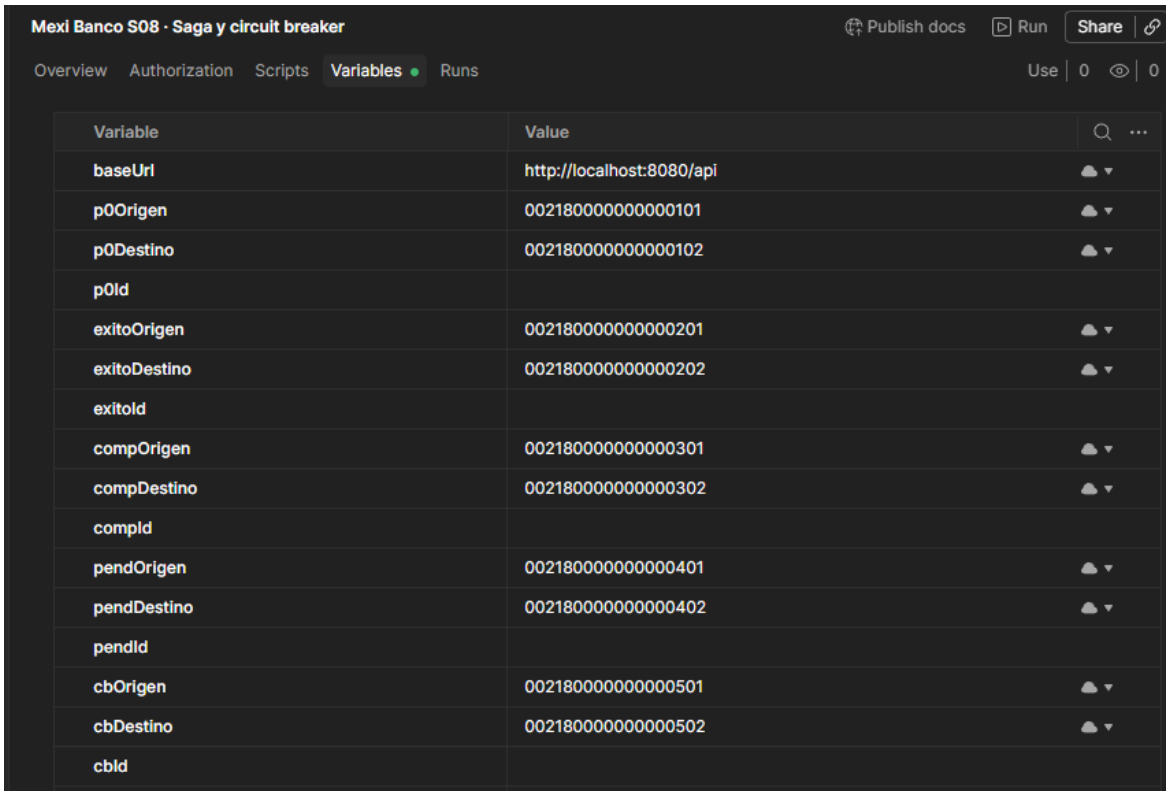


Figura 5: Variables de Postman para el gateway y los pares de cuentas de cada escenario.

También se revisó el tiempo máximo de espera de Postman. En la captura quedó configurado en 0 ms, lo que permite esperar sin que Postman cancele la petición durante la pausa de la demostración.

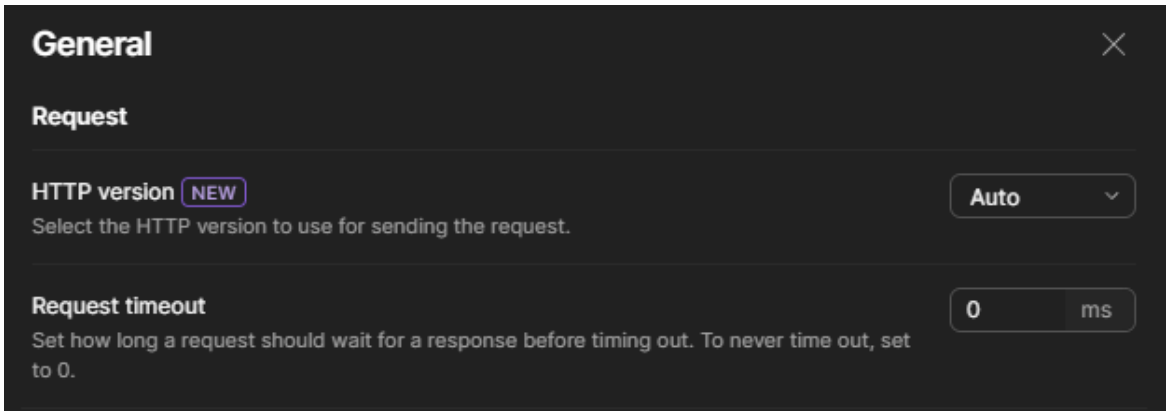


Figura 6: Request timeout de Postman configurado en 0 ms para la demostración.

1 El problema original

Primero se crearon las cuentas origen y destino con saldos iniciales de 1000 y 500, respectivamente. Después se consultaron por el gateway para corroborar que la suma inicial era de 1500. En este caso se utilizaron las cuentas con terminaciones 101 y 102.

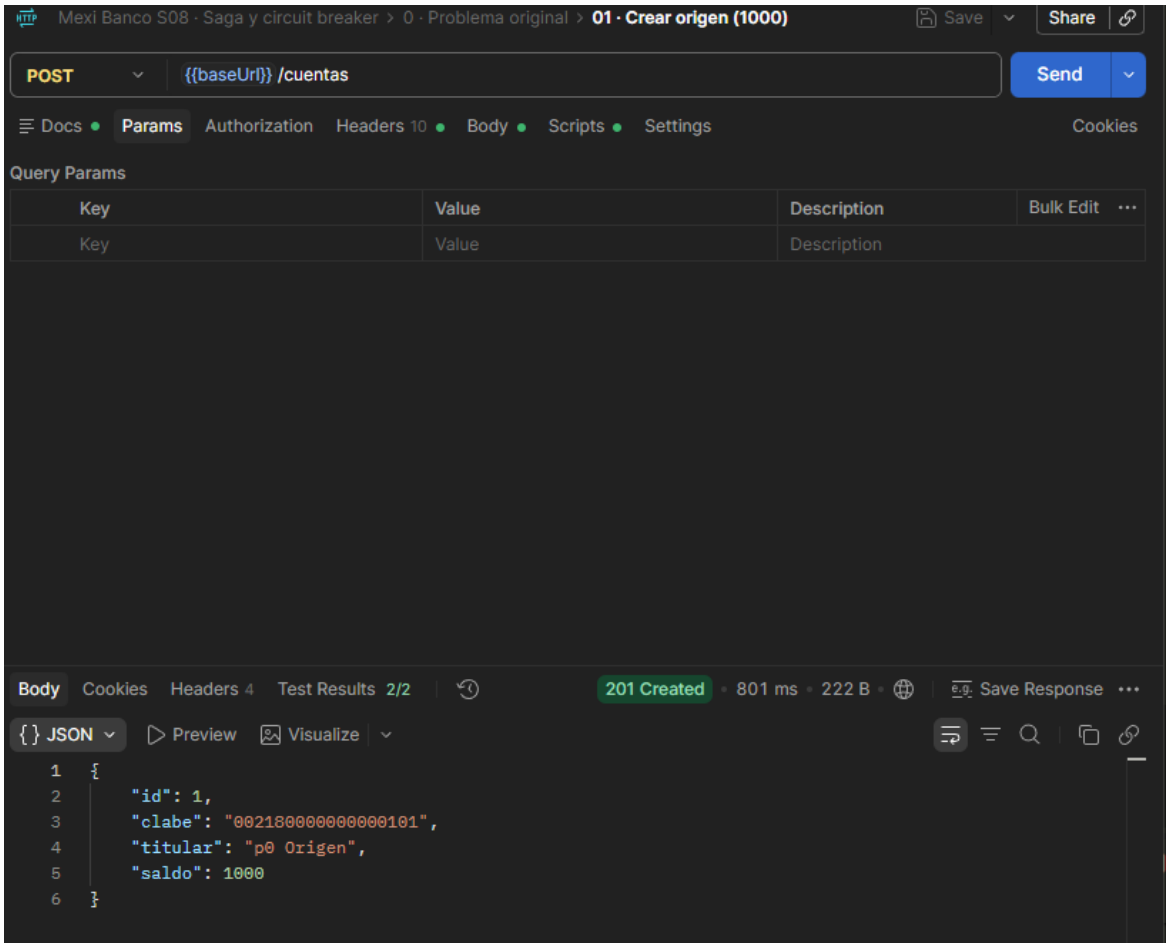


Figura 7: Creación de la cuenta origen con saldo inicial de 1000 y respuesta 201.

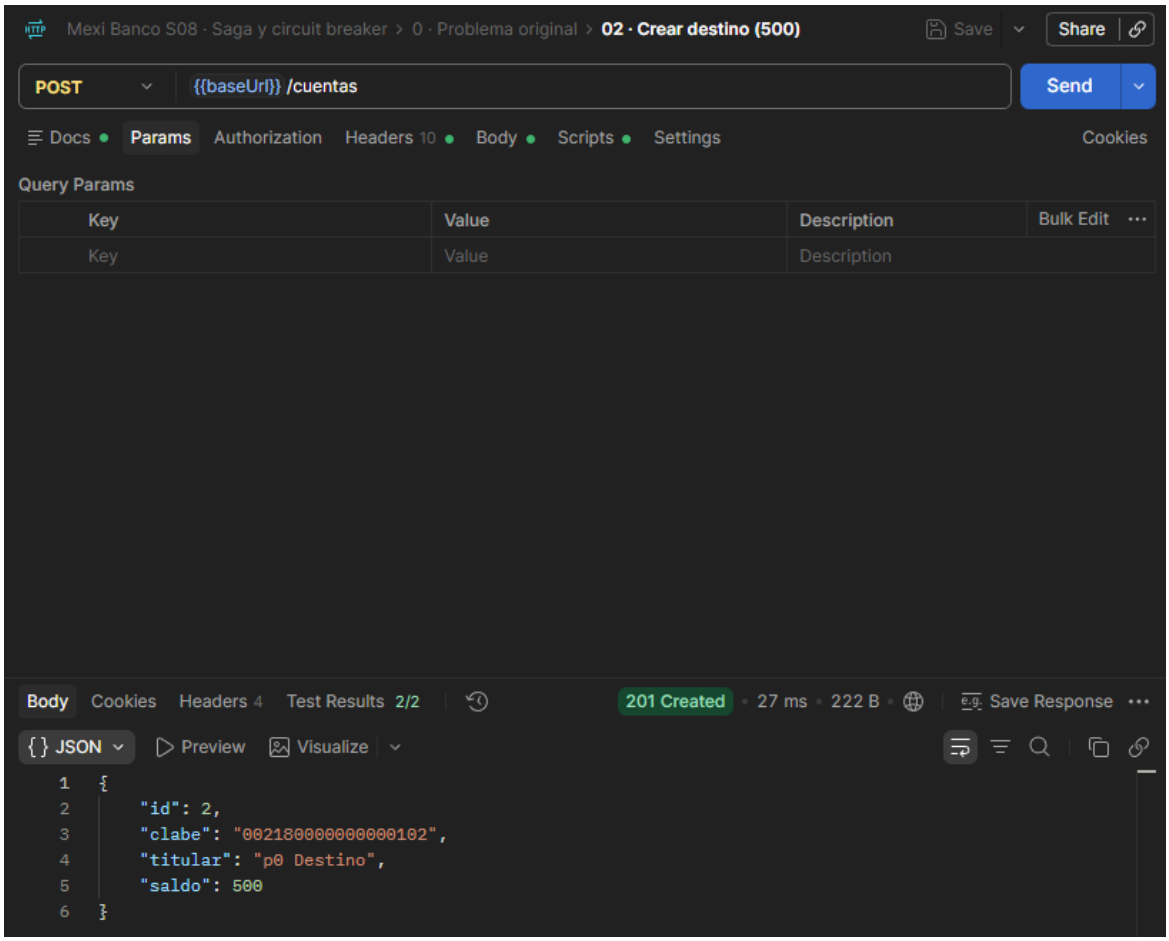


Figura 8: Creación de la cuenta destino con saldo inicial de 500 y respuesta 201.

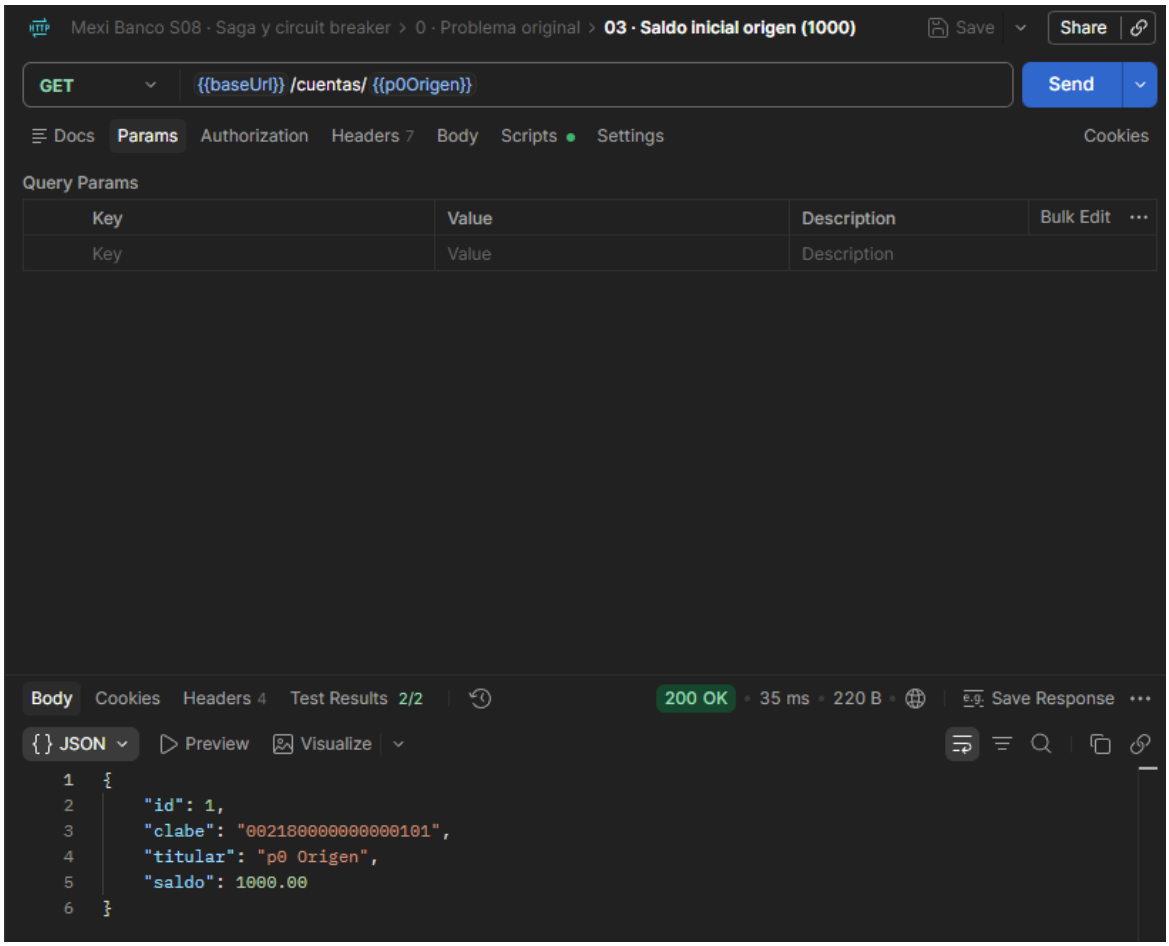


Figura 9: Consulta del saldo inicial de la cuenta origen mediante el gateway.

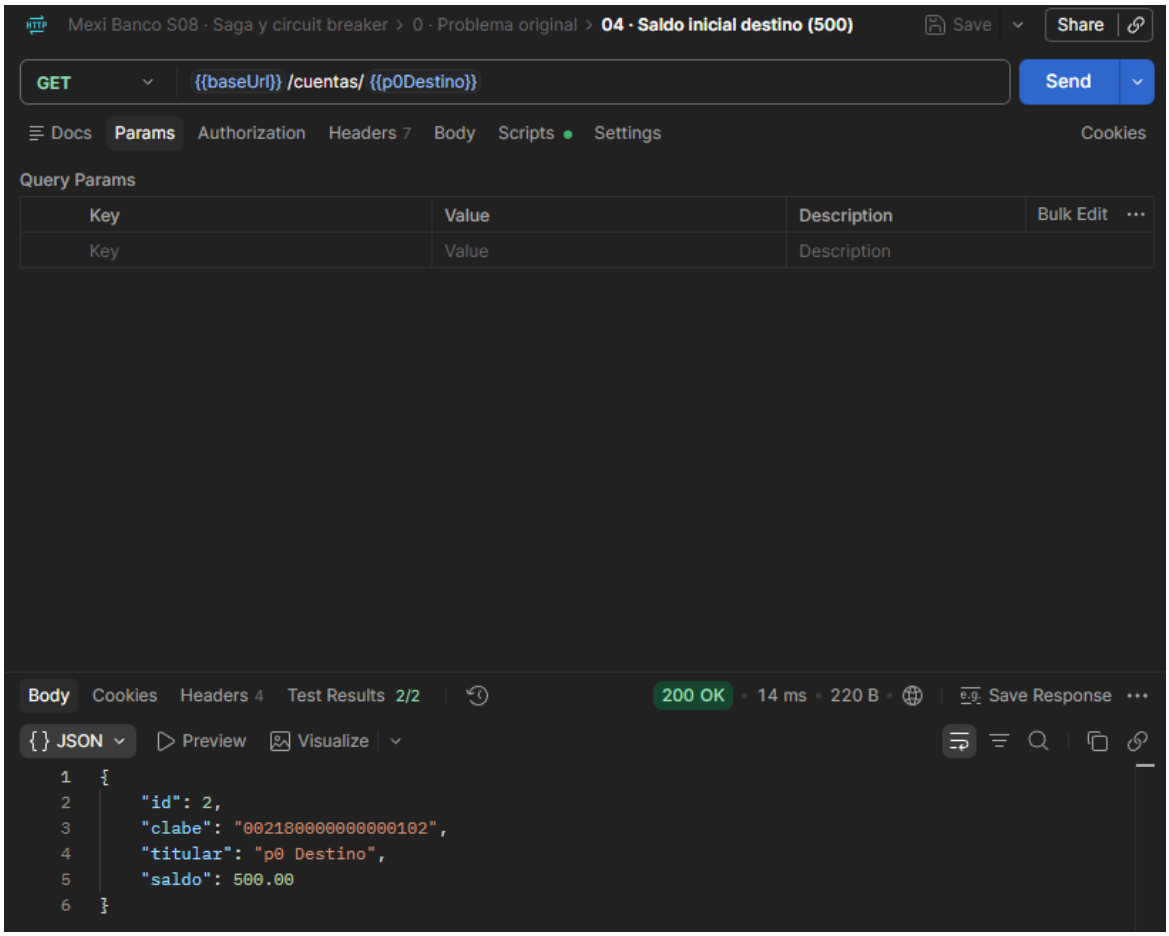


Figura 10: Consulta del saldo inicial de la cuenta destino mediante el gateway.

Ahora, con las dos cuentas listas, se estableció PAUSA_ENTRE_CARGO_Y_ABONO_MS en 20000. La pausa permite que el cargo se confirme y deja un intervalo para detener cuenta antes de que se intente el abono.

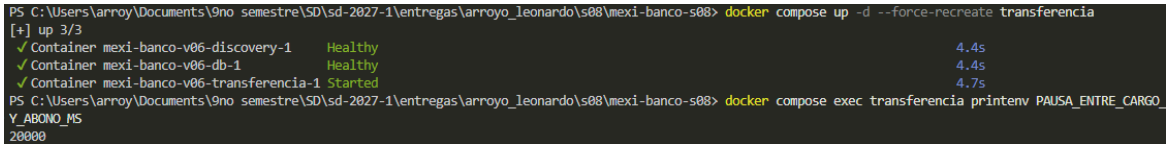


Figura 11: Configuración de la pausa de 20000 ms y comprobación de la variable dentro del contenedor.

Se envió la transferencia de 200 y se consultaron los saldos por SQL mientras la petición esperaba. Al ver 800 en el origen y 500 en el destino, se detuvo cuenta. Es decir, el cargo ya estaba confirmado, pero el destino todavía no había recibido el dinero.

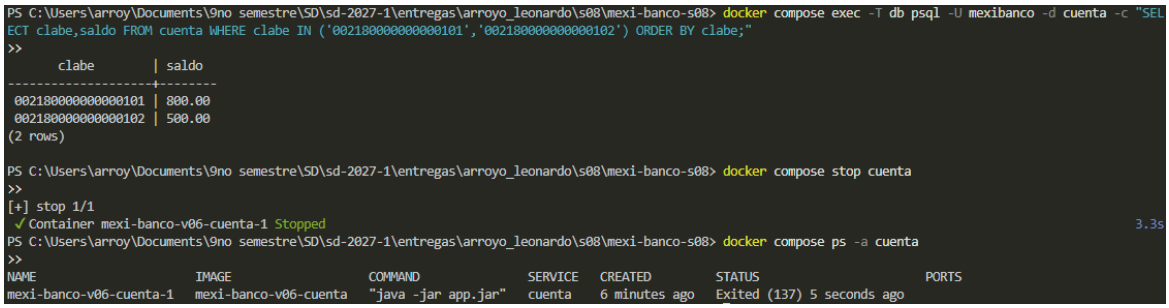


Figura 12: Consulta SQL con saldos 800 y 500 antes de detener el servicio de cuenta.

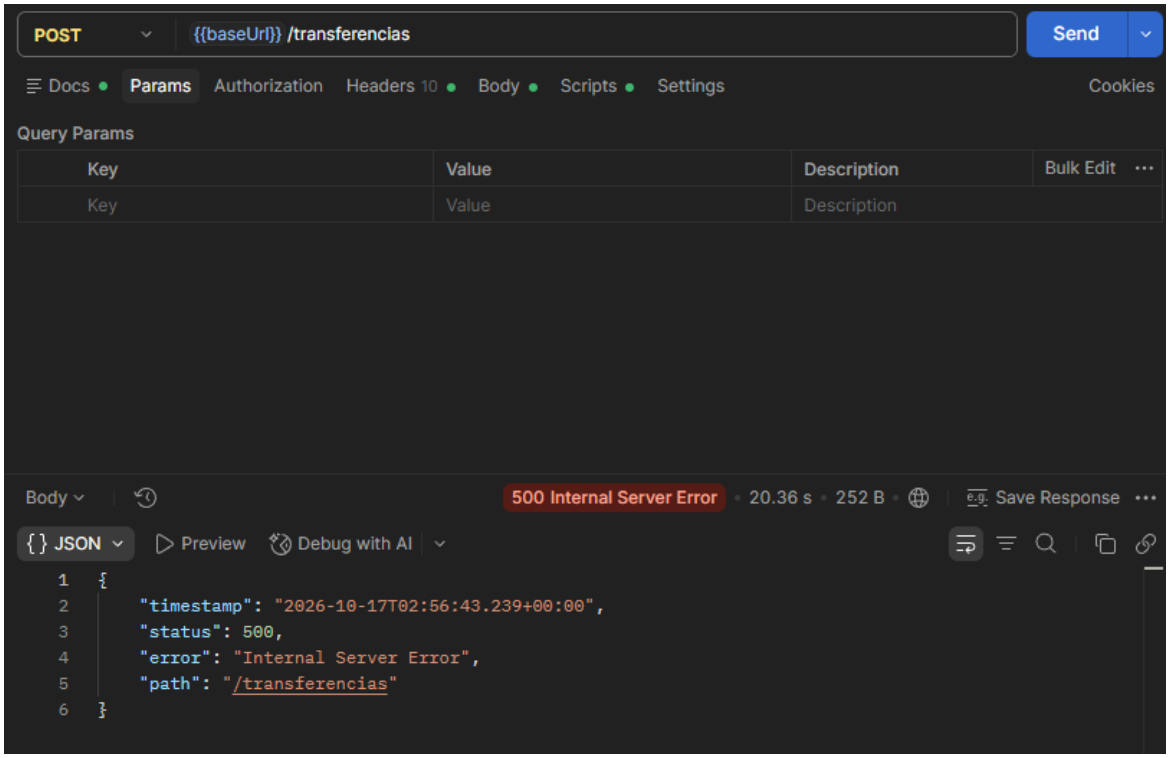


Figura 13: La transferencia original devuelve 500 Internal Server Error tras interrumpir el abono.

Después se reanudó cuenta y se volvieron a consultar los saldos. El origen continuó con 800 y el destino con 500. La suma quedó en 1300, aunque antes de comenzar había 1500; los 200 descontados no llegaron al destino.

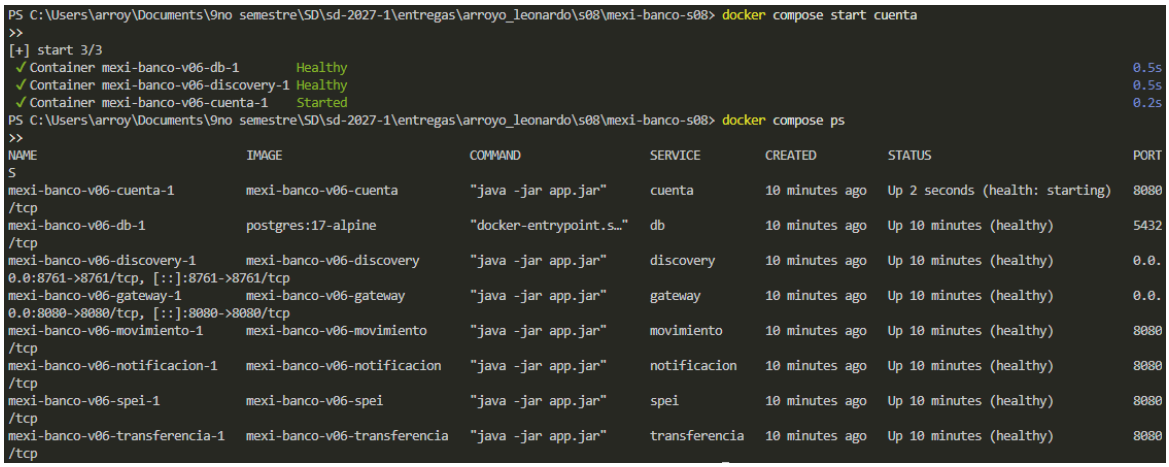


Figura 14: Reanudación del servicio de cuenta después de la falla.

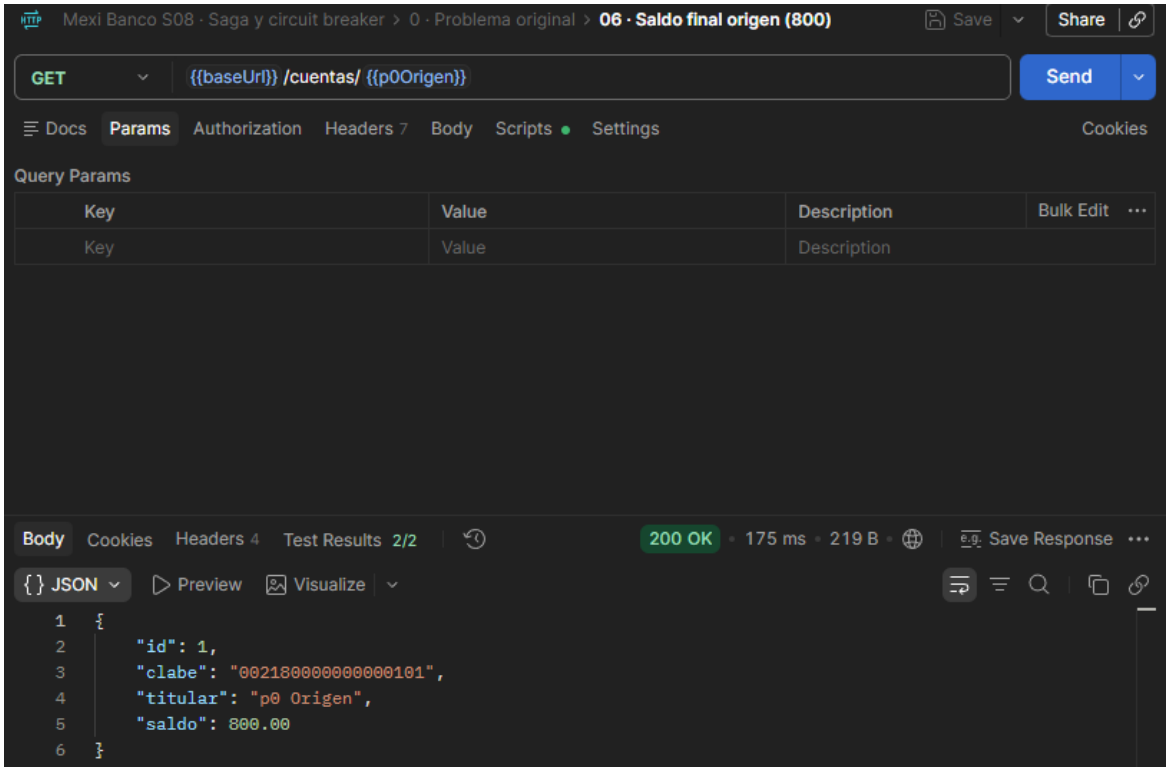


Figura 15: Saldo final del origen en el caso original: 800.

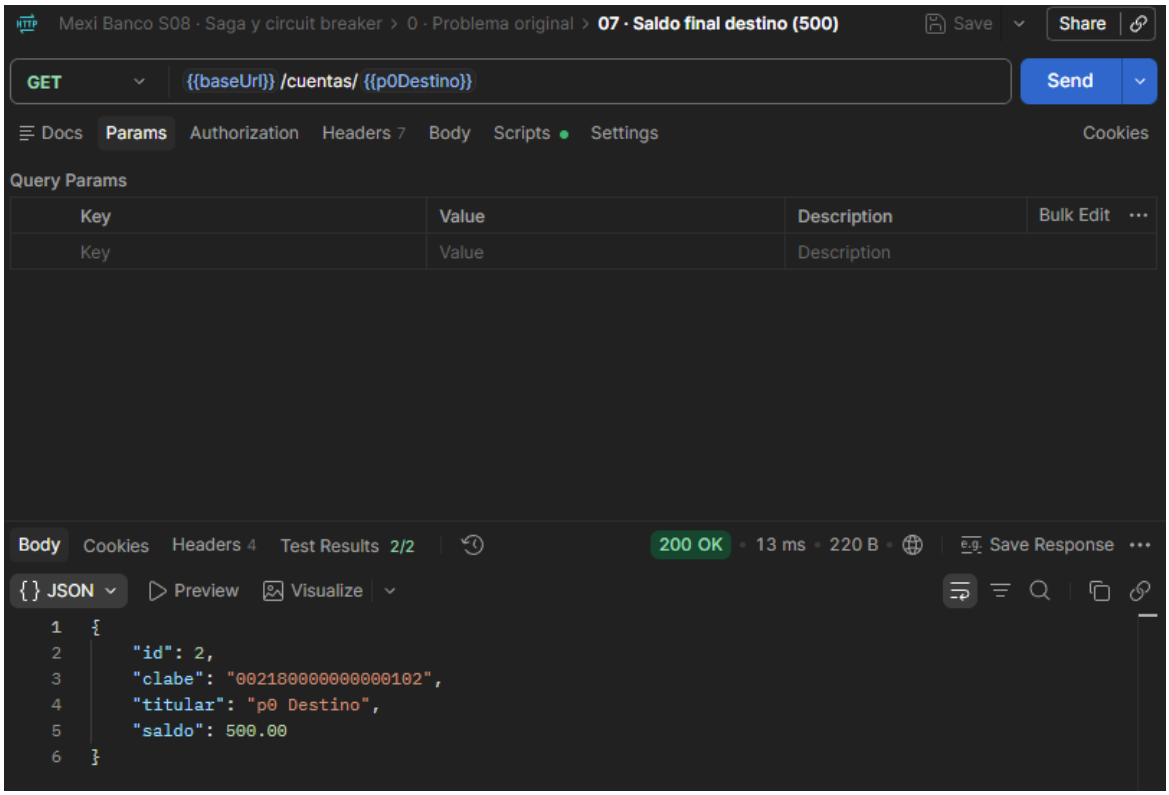


Figura 16: Saldo final del destino en el caso original: 500.

¿Qué sucedió? Cada petición HTTP confirma sus cambios en una transacción local. Cuando el servicio de cuenta cayó, el abono no pudo completarse, pero eso no deshizo el cargo que ya se había aplicado. El código original tampoco tenía una medida para devolver el dinero ni continuar la operación cuando el servicio se recuperara.

2 Diseño de la saga orquestada

Para resolverlo, TransferenciaService quedó como el orquestador. Esto significa que decide qué paso sigue, guarda el avance y, cuando el abono no se aplica, intenta devolver el monto al origen. La compensación es un nuevo abono que contrarresta el cargo anterior; no es un rollback de todas las bases de datos.

La transferencia conserva estado, detalle y fecha de actualización. El recorrido normal es INICIADA, CARGO_EN_CURSO, CARGADA, ABONO_EN_CURSO, ABONADA y COMPLETADA. Los estados EN_CURSO se guardan antes de llamar a otro servicio para que una interrupción deje una referencia de dónde se quedó la operación.

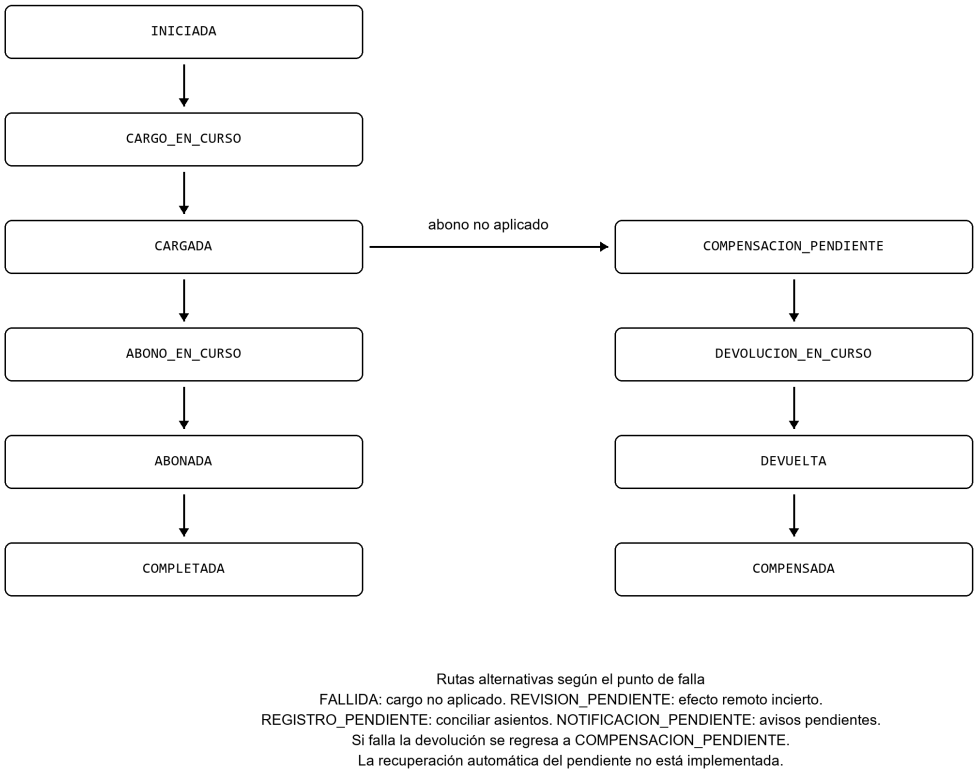


Figura 17: Diagrama de estados de la saga y rutas de compensación.

Si falla la consulta inicial, no se intenta el cargo y se marca FALLIDA. Si el cargo fue aplicado, pero el abono fue rechazado o se sabe que no se envió, se inicia la devolución. Si una respuesta de cargo o abono es incierta, se guarda REVISION_PENDIENTE: un timeout no demuestra por sí solo que el otro servicio no haya confirmado el cambio.

Al devolver el monto al origen se registra DEVUELTA. Después se añade el movimiento de compensación y se termina en COMPENSADA. Si la devolución no se confirma, queda COMPENSACION_PENDIENTE. Si el dinero ya llegó al destino, una falla posterior en los movimientos o avisos deja REGISTRO_PENDIENTE o NOTIFICACION_PENDIENTE; no sería correcto devolver dinero que el destinatario ya recibió.

3 Código clave y explicación

El primer cambio fue agregar EstadoTransferencia y los campos de seguimiento en Transferencia. El estado se guarda como texto, con EnumType.STRING. Así la tabla conserva un valor entendible, como COMPENSADA, en vez de depender de la posición numérica del enum.

```
@Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 40)
    private EstadoTransferencia estado;
    @Column(nullable = false)
    private Instant actualizada;
    @Column(length = 2000)
    private String detalle;
```

Cada cambio de estado se confirma con saveAndFlush y también se escribe en los logs. El método transferir no lleva una transacción global de Spring; de esta manera, las transiciones locales se confirman antes de realizar el siguiente salto HTTP. Esto conserva el estado actual en la fila, aunque no sustituye un historial completo de todos los pasos.

```
private Transferencia estado(Transferencia t, EstadoTransferencia nuevo, String detalle) {
        t.cambiarEstado(nuevo, detalle);
        transferencias.saveAndFlush(t);
        log.info("SAGA id={} estado={} detalle={}", t.getId(), nuevo, detalle);
        return t;
    }
```

Para provocar la falla sin apagar cuenta se utilizó FALLAR_ABONO_CLABE. Cuando el destino coincide con esa CLABE, el orquestador entra a compensar antes de enviar el abono al destino. De esta forma se sabe que el destino no recibió el dinero y la devolución puede probarse con cuenta disponible.

```
pausa(t);
        if (!fallarAbonoClabe.isBlank() && fallarAbonoClabe.equals(destino)) {
            return compensar(t, "Falla de abono simulada para " + destino, true);
        }
```

El método compensar guarda primero que la devolución está pendiente y después que está en curso. Si la llamada para abonar al origen falla, vuelve a dejar el pendiente con un detalle que pide conciliar antes de reintentar.

```
private Transferencia compensar(Transferencia t, String causa, boolean cargoRegistrado) {
        estado(t, COMPENSACION_PENDIENTE, causa + "; devolución pendiente");
        estado(t, DEVOLUCION_EN_CURSO, causa + "; se intentará devolver al origen");
        CuentaRemota devuelta;
        try {
            devuelta = cuentas.abonar(t.getClaveOrigen(), t.getMonto());
        } catch (RuntimeException error) {
            return estado(t, COMPENSACION_PENDIENTE,
                    causa + "; devolución NO confirmada; conciliar antes de reintentar: " + mensaje(error))
     ;
        }
```

Después de confirmar el abono de devolución, el código guarda DEVUELTA y registra un movimiento positivo con la referencia COMPENSACION saga seguida del identificador. Si ese asiento falla, el detalle avisa que el dinero ya fue devuelto. Con esto se evita confundir un problema del registro con la necesidad de abonar otra vez.

```
estado(t, DEVUELTA, causa + "; dinero devuelto, movimiento pendiente");
        try {
            movimientos.registrar(t.getClaveOrigen(), TipoMovimiento.TRANSFERENCIA_RECIBIDA,
                    t.getMonto(), devuelta.saldo(), "COMPENSACION saga " + t.getId());
        } catch (RuntimeException error) {
            return estado(t, REGISTRO_PENDIENTE,
                    "DINERO YA DEVUELTO; no abonar otra vez. Conciliar movimiento: " + mensaje(error));
        }
        if (!cargoRegistrado) {
            return estado(t, REGISTRO_PENDIENTE,
                    "DINERO YA DEVUELTO y compensación registrada; conciliar asiento de cargo");
        }
        return estado(t, COMPENSADA, causa + "; monto devuelto al origen y movimiento registrado");
    }
```

El circuito se colocó en CuentaCliente, envolviendo las operaciones de consulta, cargo y abono. En el siguiente fragmento se ve cómo la operación HTTP pasa por circuito.ejecutar antes de contactar a cuenta.

```
public CuentaRemota abonar(String clabe, BigDecimal monto) {
        return circuito.ejecutar("abonar", () -> http.post()
                .uri("/cuentas/{clabe}/abonos", clabe).body(Map.of("monto", monto))
                .retrieve().body(CuentaRemota.class));
    }
```

El circuit breaker se hizo a mano. Cuenta tres fallas de infraestructura consecutivas y permanece abierto al menos 15 segundos. En ese estado rechaza nuevas operaciones sin ejecutar HTTP. Después del plazo, una petición puede entrar como prueba semiabierta; si funciona, cierra el circuito, y si falla, vuelve a abrirlo. Los errores de negocio como 404 y 422 no se cuentan como caída del servicio.

```
private synchronized long admitir(String operacion) {
        if (estado == Estado.ABIERTO && reloj.getAsLong() - abiertoDesde >= esperaNanos) {
            transicion(Estado.SEMIABIERTO);
            log.info("HTTP_CALL cuenta operacion={} estado={} prueba=unica", operacion, estado);
            return generacion;
        }
        if (estado != Estado.CERRADO) {
            log.info("RECHAZO_SIN_HTTP cuenta operacion={} estado={}", operacion, estado);
            throw new CircuitoAbiertoException("Circuito cuenta " + estado + "; sin llamada HTTP");
        }
        log.info("HTTP_CALL cuenta operacion={} estado={}", operacion, estado);
        return generacion;
    }
```

El contador y el tiempo de espera pertenecen a esta instancia de transferencia y viven en memoria. Si el proceso se reinicia, el circuito comienza de nuevo cerrado. El estado de cada transferencia sí se conserva en su base, por lo que son dos mecanismos de seguimiento distintos.

Además, el cliente HTTP tiene un timeout de conexión de 2 segundos y uno de lectura de 5 segundos. Estos tiempos limitan cada llamada saliente; el timeout de 60 segundos del gateway sólo permite observar la pausa completa de la demostración.

```
@Bean
        RestClientCustomizer tiemposMaximos() {
                SimpleClientHttpRequestFactory fabrica = new SimpleClientHttpRequestFactory();
                fabrica.setConnectTimeout(2_000);
                fabrica.setReadTimeout(5_000);
                return builder -> builder.requestFactory(fabrica);
        }
```

El controlador también explica el resultado con su código HTTP y con el estado de la saga. Una transferencia COMPLETADA responde 201; una COMPENSADA responde 409, porque la transferencia solicitada no terminó aunque el dinero se haya devuelto. Los estados fallidos o pendientes responden 503 y conservan el detalle de lo ocurrido.

4 Transferencia exitosa

Después de los cambios se realizó una transferencia de 200 con las cuentas de terminación 201 y 202. En este caso no se provocó ninguna falla. Postman respondió 201 Created y la transferencia quedó COMPLETADA, con identificador 1.

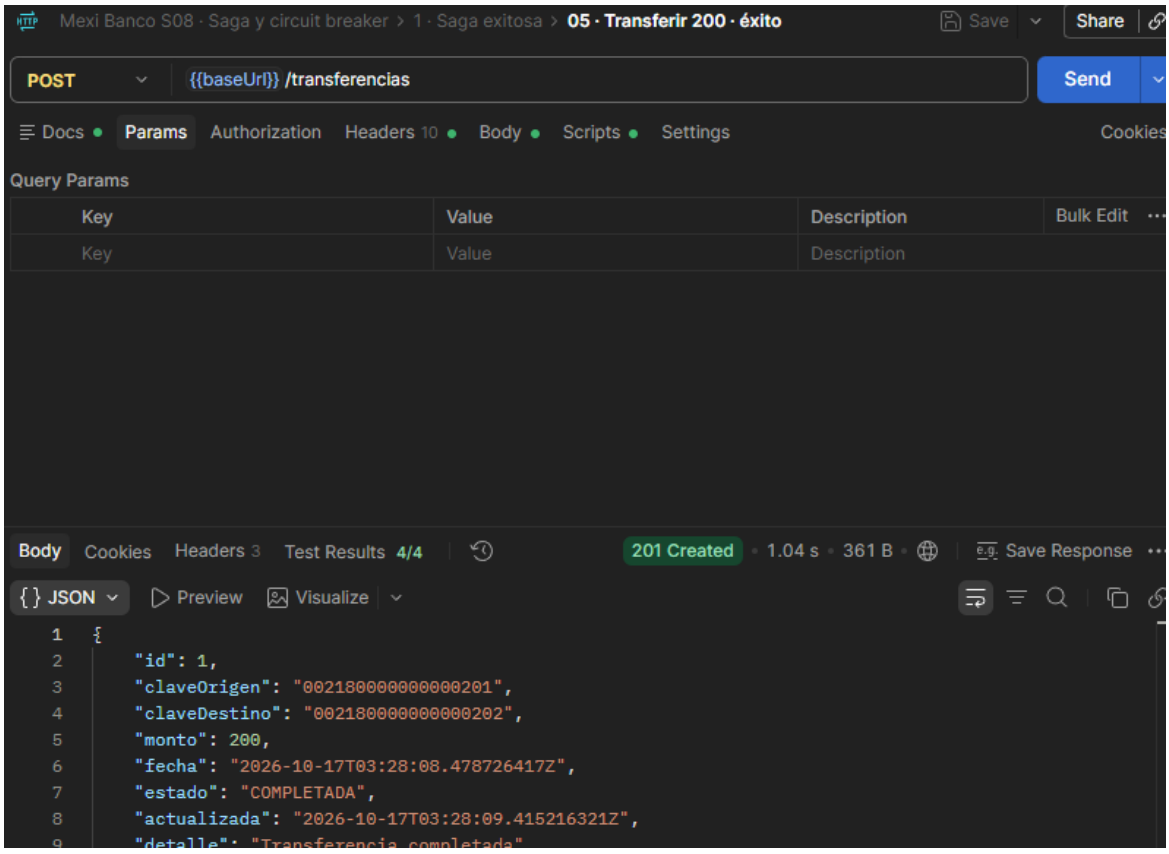


Figura 18: Respuesta de la saga exitosa con estado COMPLETADA e identificador 1.

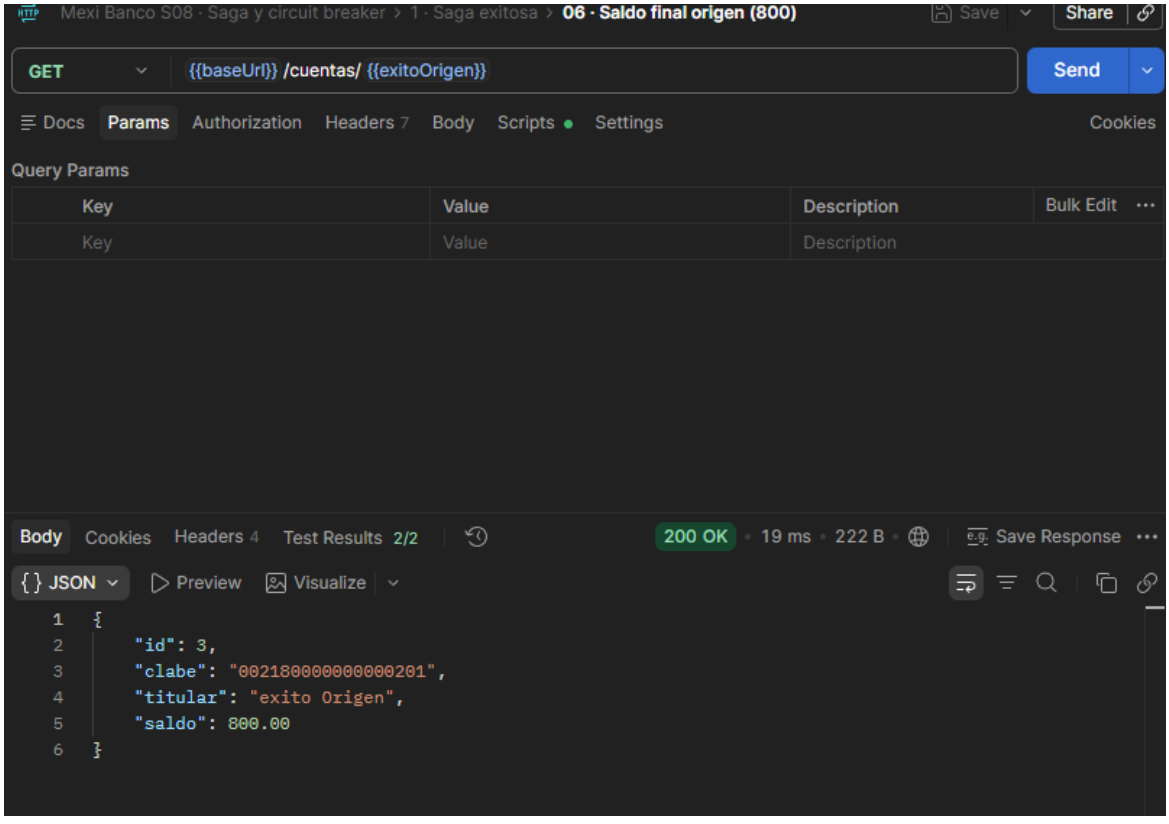


Figura 19: Saldo final del origen después de transferir 200: 800.

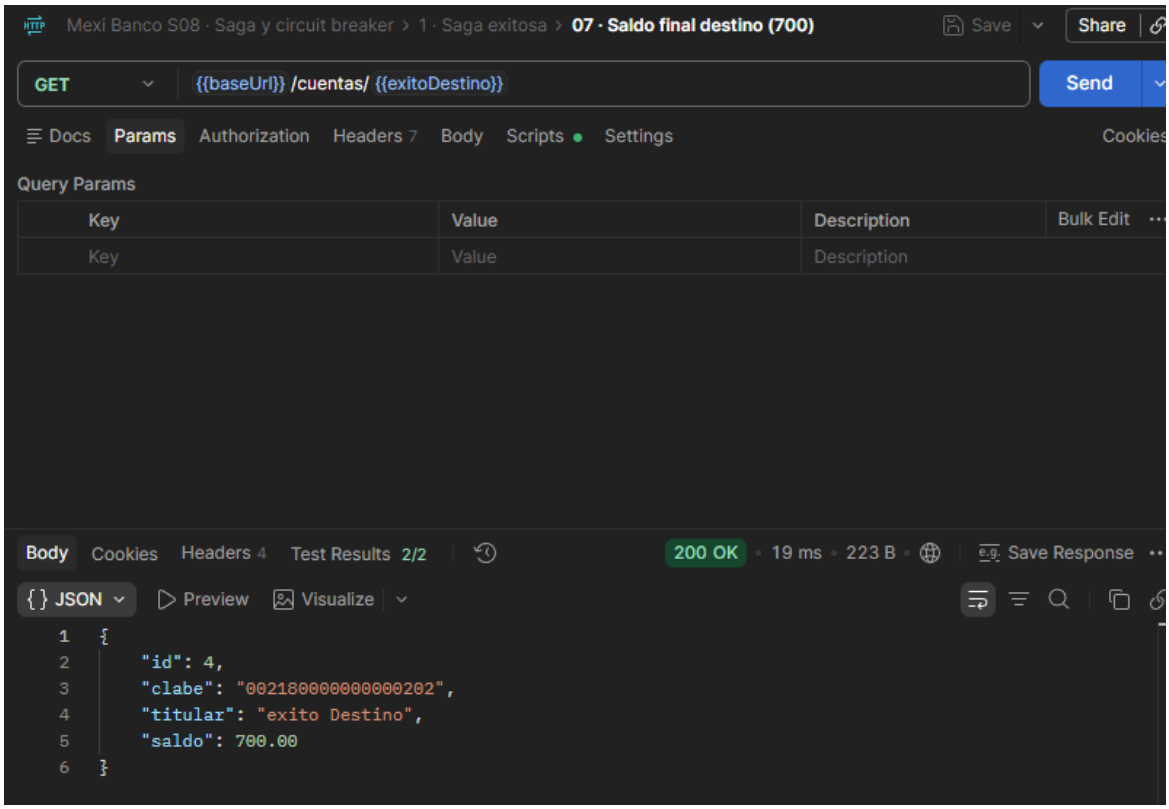


Figura 20: Saldo final del destino después de recibir 200: 700.

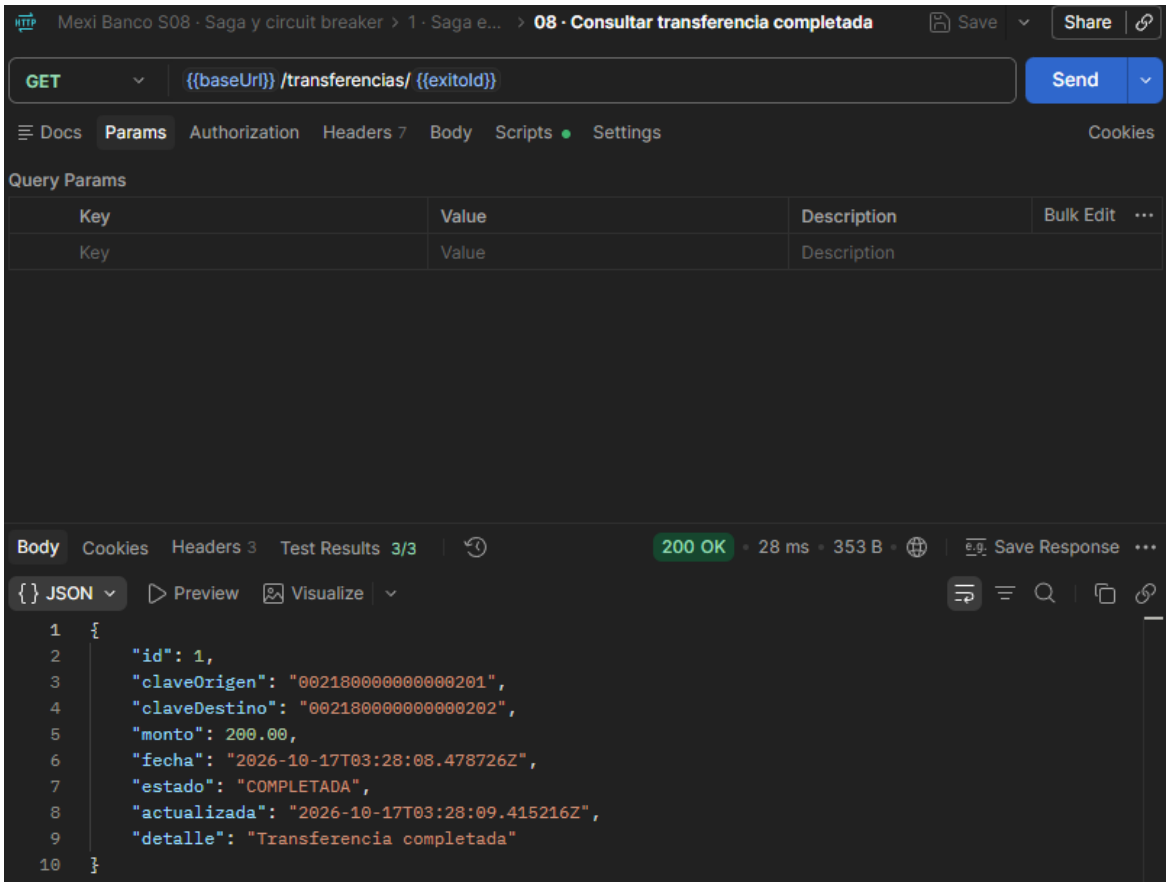


Figura 21: Consulta posterior de la transferencia 1 con su estado COMPLETADA.

Con esto se comprobó el recorrido normal: los 200 descontados al origen llegaron al destino y la suma se mantuvo en 1500. La consulta posterior devuelve el mismo estado guardado por el servicio.

5 Compensación funcionando

Para este caso se eligió la cuenta destino con terminación 302 como FALLAR_ABONO_CLABE y se dejó la pausa en 0. Se utilizaron cuentas nuevas con saldos iniciales de 1000 y 500 para apreciar la devolución de los 200.

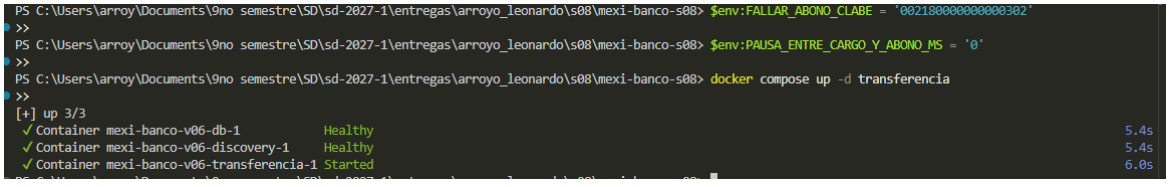


Figura 22: Configuración de la falla simulada del abono hacia la CLABE del escenario de compensación.

Al enviar la transferencia apareció 409 Conflict, pero el cuerpo de la respuesta explicó el motivo: la falla del abono fue simulada, el monto se devolvió al origen y se registró el movimiento. La transferencia 2 terminó COMPENSADA.

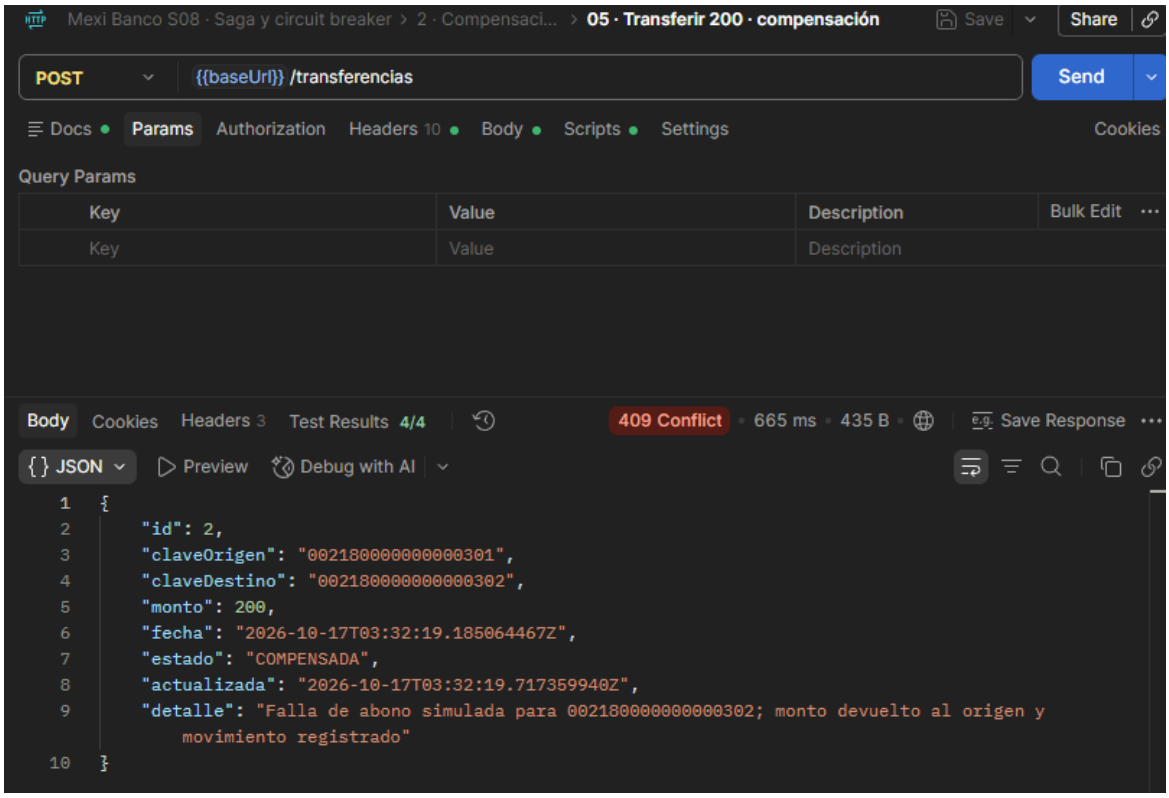


Figura 23: Respuesta 409 con estado COMPENSADA y detalle de la devolución realizada.

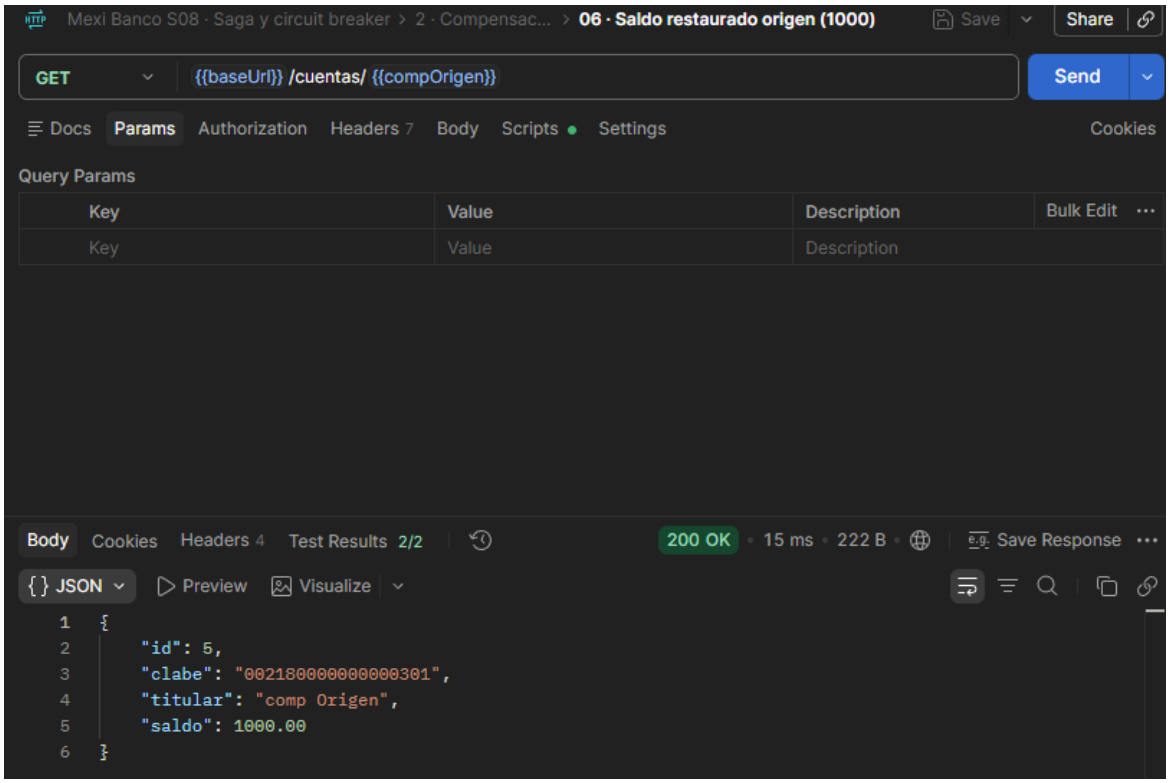


Figura 24: El origen recuperó su saldo inicial de 1000.

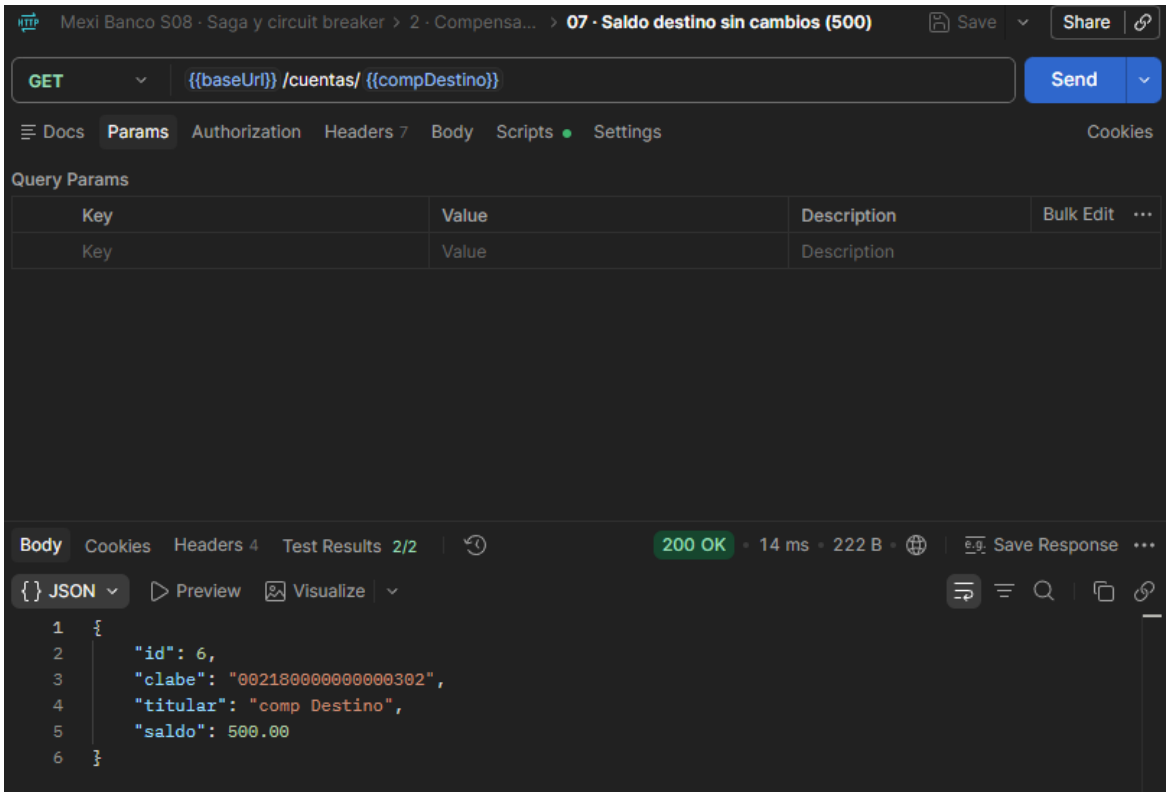


Figura 25: El destino permaneció con saldo de 500 porque no se envió el abono.

Ahora bien, no bastaba con revisar los saldos. También se consultaron los movimientos del origen y se observó el cargo por 200 y el abono de compensación por el mismo monto. Esto deja escrito por qué el saldo volvió a 1000, en vez de borrar el cargo como si nunca hubiera pasado.

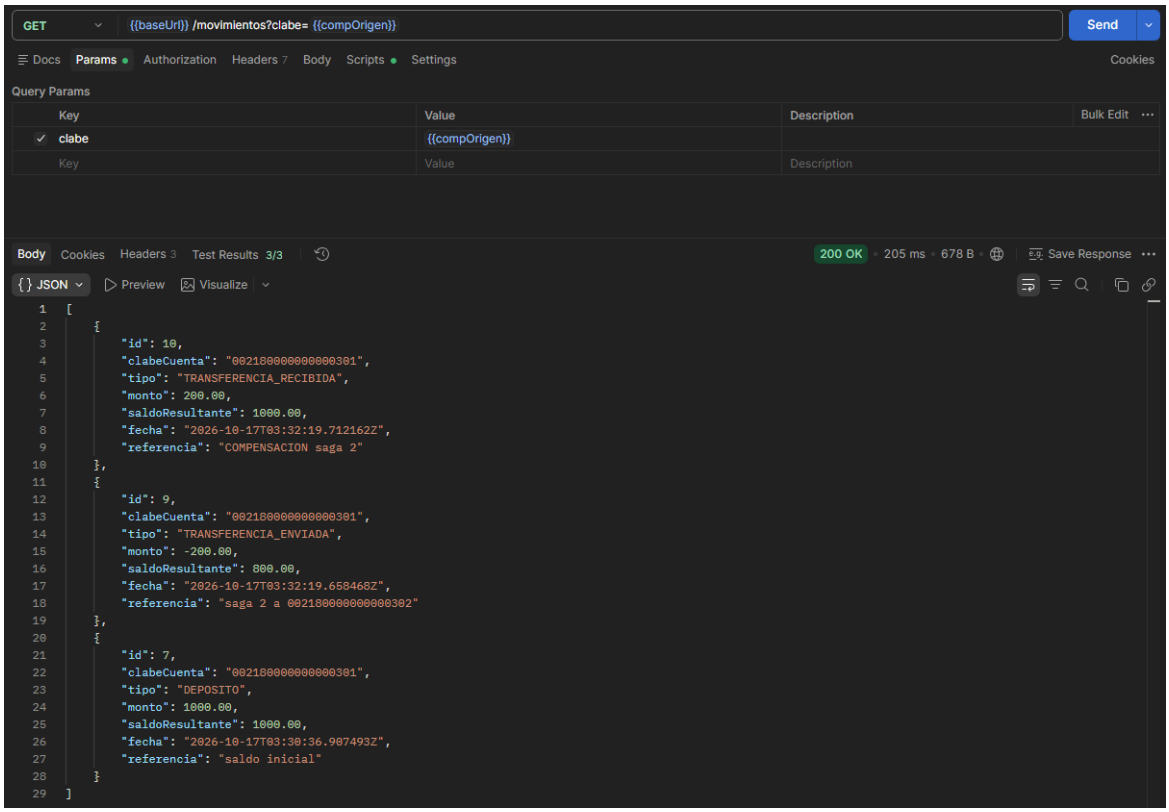


Figura 26: Movimientos del origen con cargo y devolución identificada como COMPENSACION saga 2.

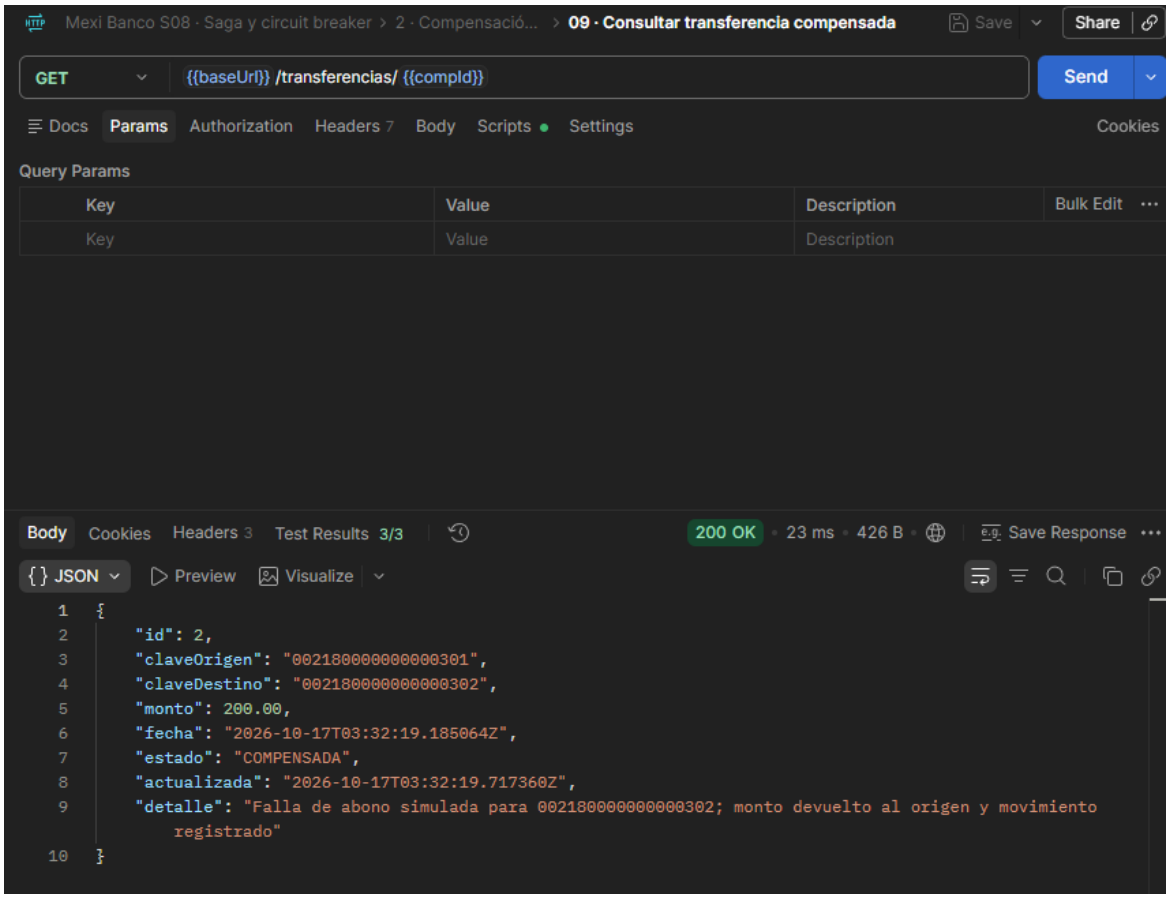


Figura 27: Consulta de la transferencia 2 con estado final COMPENSADA.

La consulta posterior por identificador muestra el estado que el servicio recupera de su repositorio. El código de persistencia permite relacionar esta respuesta con la fila guardada de la transferencia; las capturas de este caso muestran la consulta por API y los movimientos.

6 Cuando la compensación también falla

Después se repitió la interrupción con las cuentas de terminación 401 y 402. Se configuró una pausa de 20000 ms y la falla simulada hacia el destino 402. Mientras la petición esperaba se detuvo cuenta, de modo que el abono al destino se omitió por la falla controlada y el intento de devolución tampoco pudo atenderse.

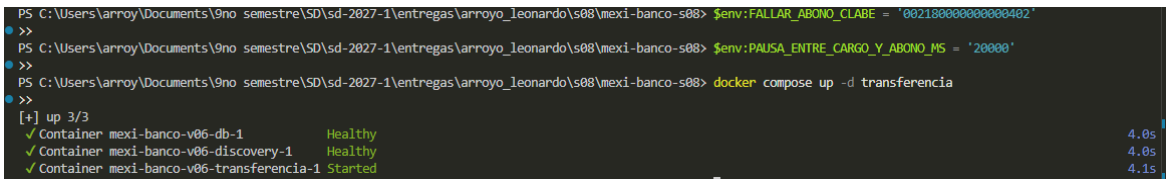


Figura 28: Configuración de la falla simulada y de la pausa para probar una devolución pendiente.

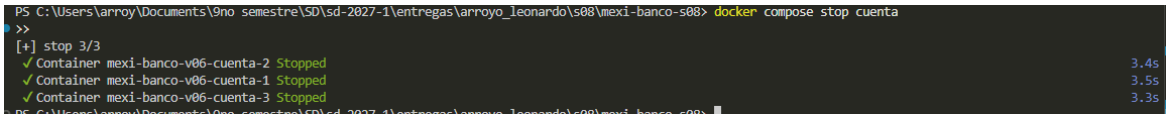


Figura 29: Detención de las instancias de cuenta mientras la transferencia estaba en la pausa.

Esta vez la respuesta fue 503 Service Unavailable y la transferencia 3 quedó en COMPENSACION_PENDIENTE. El detalle indicó que la devolución no estaba confirmada porque no había instancias disponibles de cuenta. Aunque la transferencia no pudo terminar, ahora quedó registrado el trabajo pendiente.

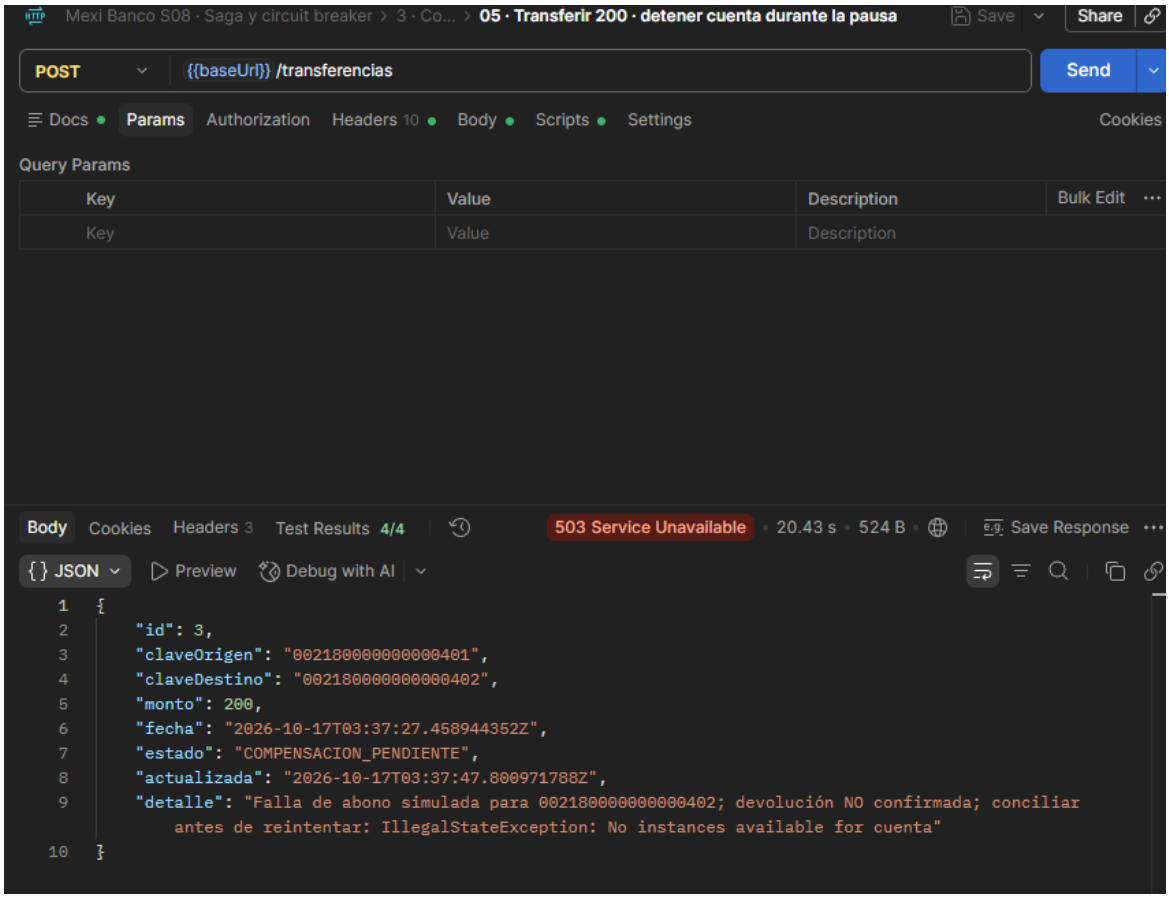


Figura 30: Respuesta 503 con identificador 3 y estado COMPENSACION_PENDIENTE.

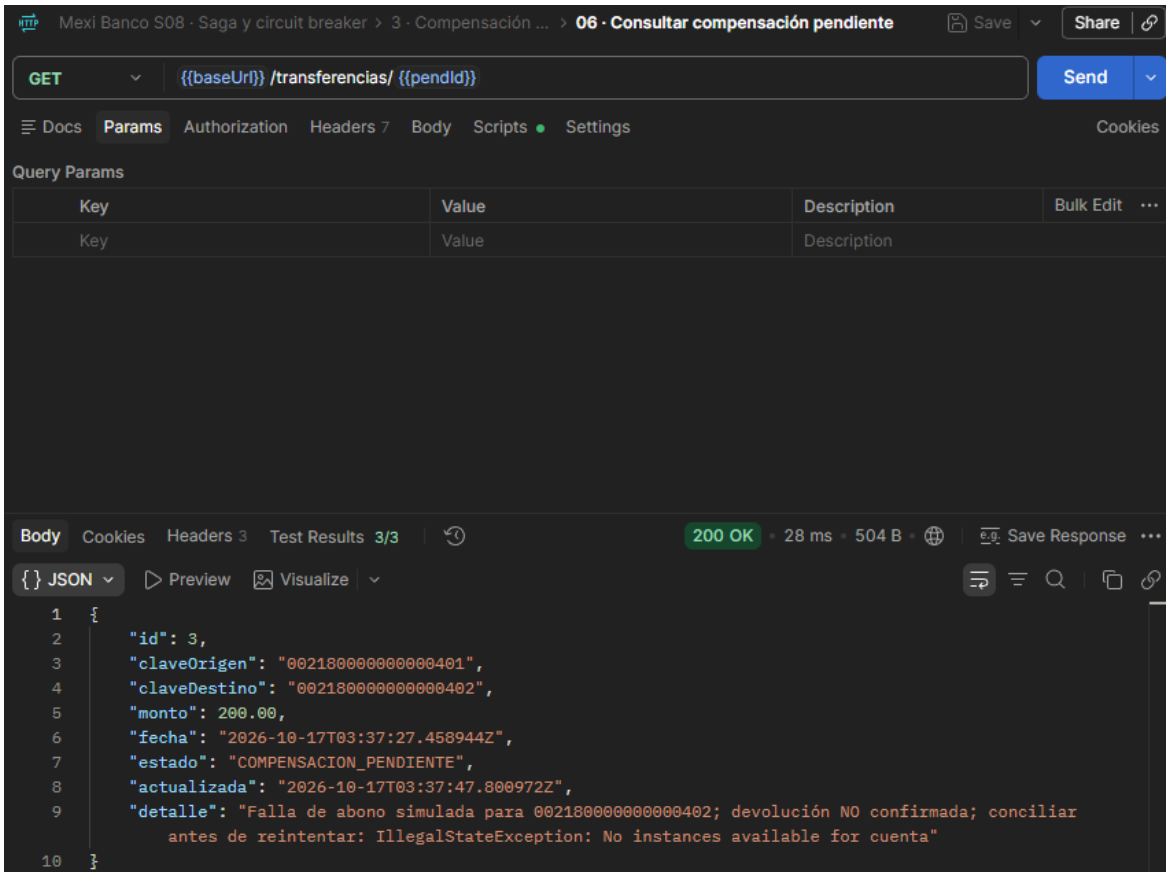


Figura 31: Consulta de la transferencia pendiente mientras cuenta seguía detenida.

Al recuperar cuenta se consultaron otra vez los saldos. El origen siguió con 800 y el destino con 500. Puede parecer el mismo resultado económico del primer caso, pero la diferencia es que la transferencia conserva el pendiente y explica que la devolución no se confirmó.

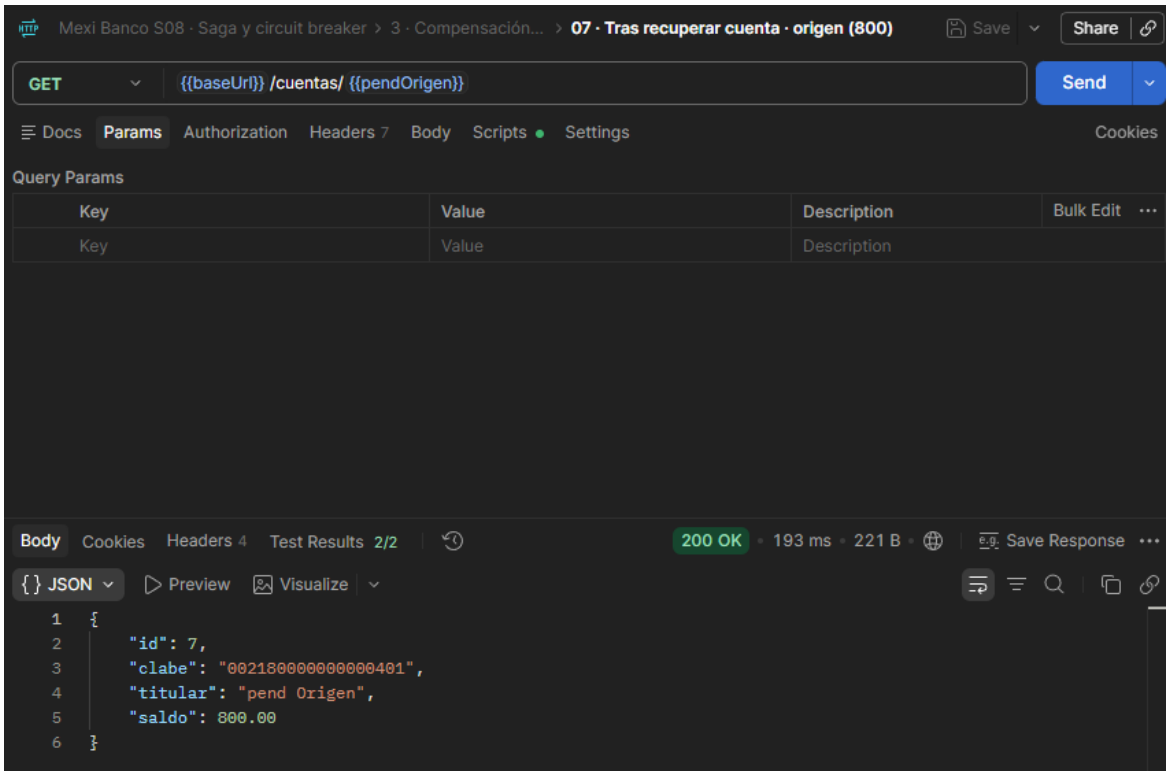


Figura 32: Saldo del origen después de recuperar cuenta: continúa en 800.

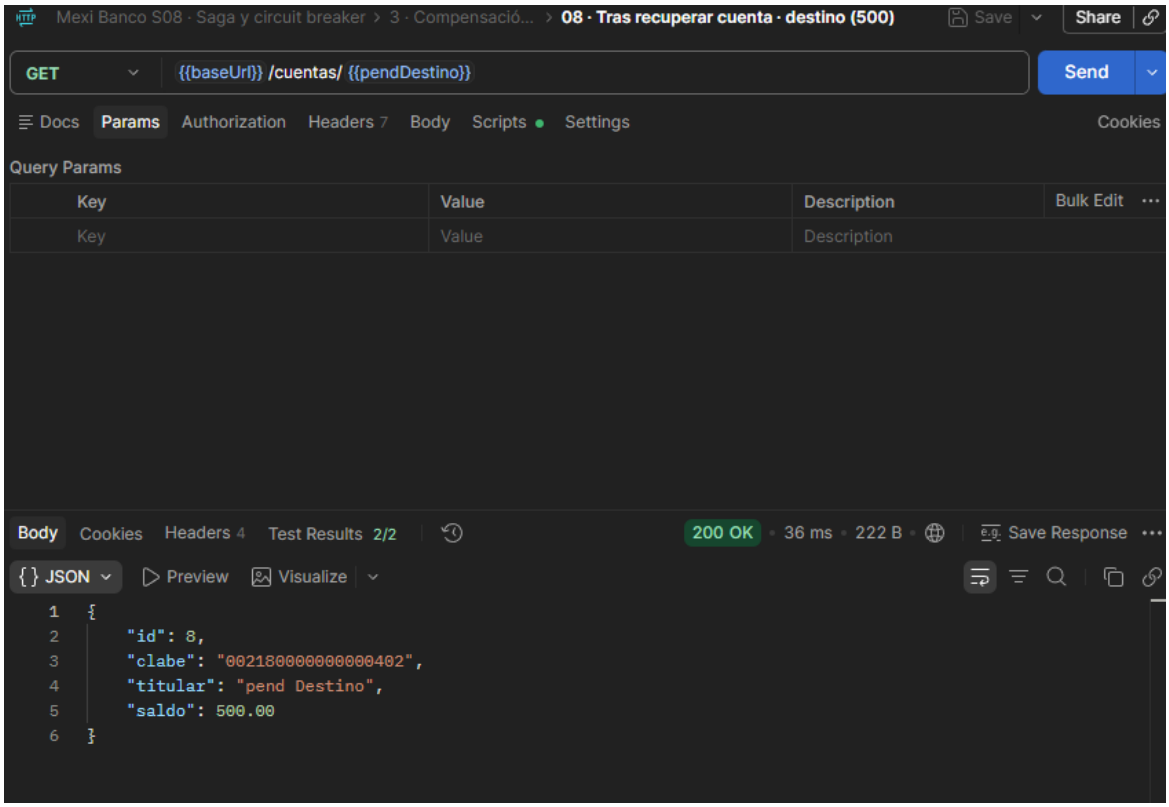


Figura 33: Saldo del destino después de recuperar cuenta: continúa en 500.

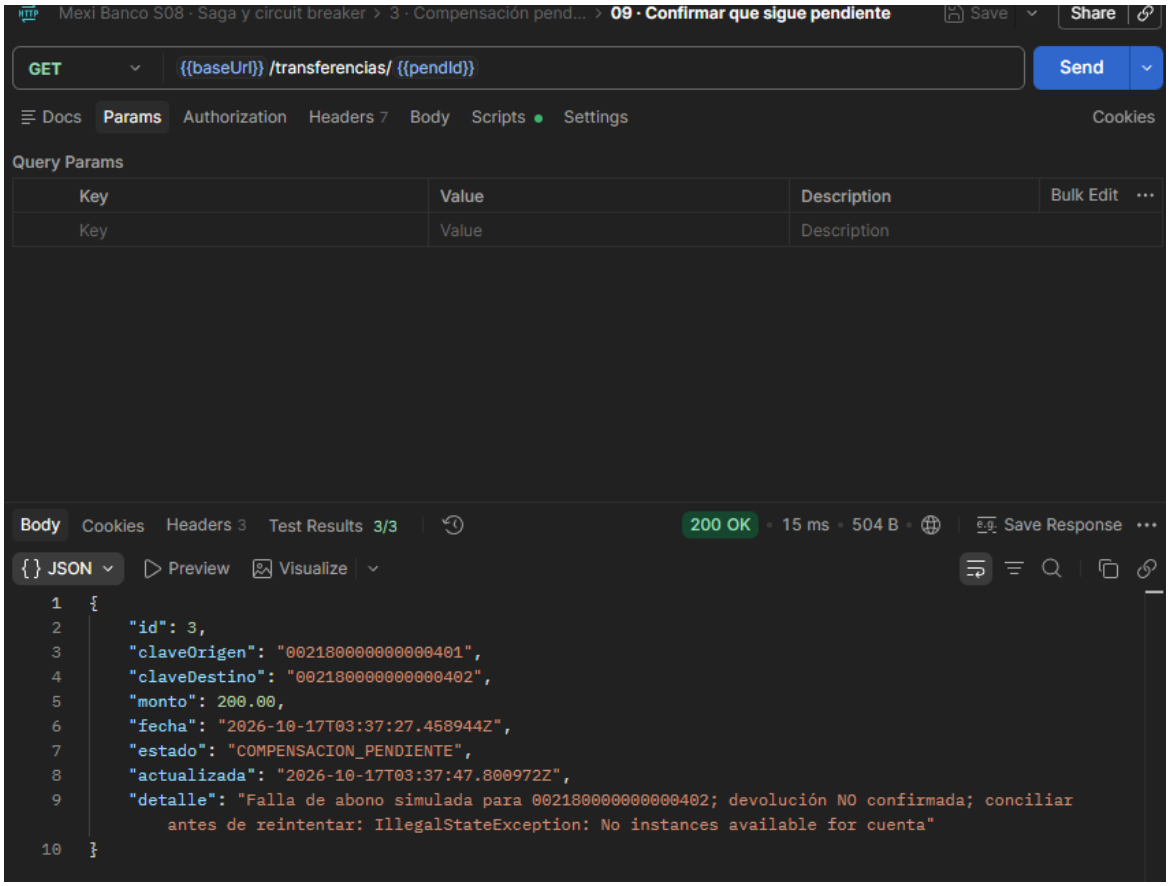


Figura 34: La transferencia 3 sigue pendiente después de la recuperación del servicio.

¿Entonces qué pasó con esto? No hay un scheduler que busque y termine las compensaciones pendientes. Encender cuenta permite que vuelva a recibir peticiones, pero no reenvía automáticamente la devolución. En esta versión la recuperación todavía necesita un proceso que concilie el intento anterior y continúe la operación.

Para terminarla de forma segura, cuenta tendría que aceptar una clave estable, por ejemplo transferencia:3:compensacion, y guardarla junto con el cambio de saldo en una misma transacción local, usando una restricción única. Si llega otra vez la misma clave, debe devolver el resultado anterior sin volver a sumar los 200. Eso es lo que hace idempotente a la compensación.

Sólo después convendría programar reintentos con espera creciente. Los movimientos también necesitarían deduplicación. Una bandera guardada únicamente en transferencia no basta: la devolución pudo haberse confirmado en cuenta y perderse la respuesta por la red. Si se reintentara sin verificar, se podría devolver dos veces.

7 Circuit breaker manual

Primero se hizo una transferencia de 10 con el circuito cerrado. La petición respondió 201 y terminó COMPLETADA. En los logs aparecen las llamadas HTTP permitidas y las transiciones normales de la saga.

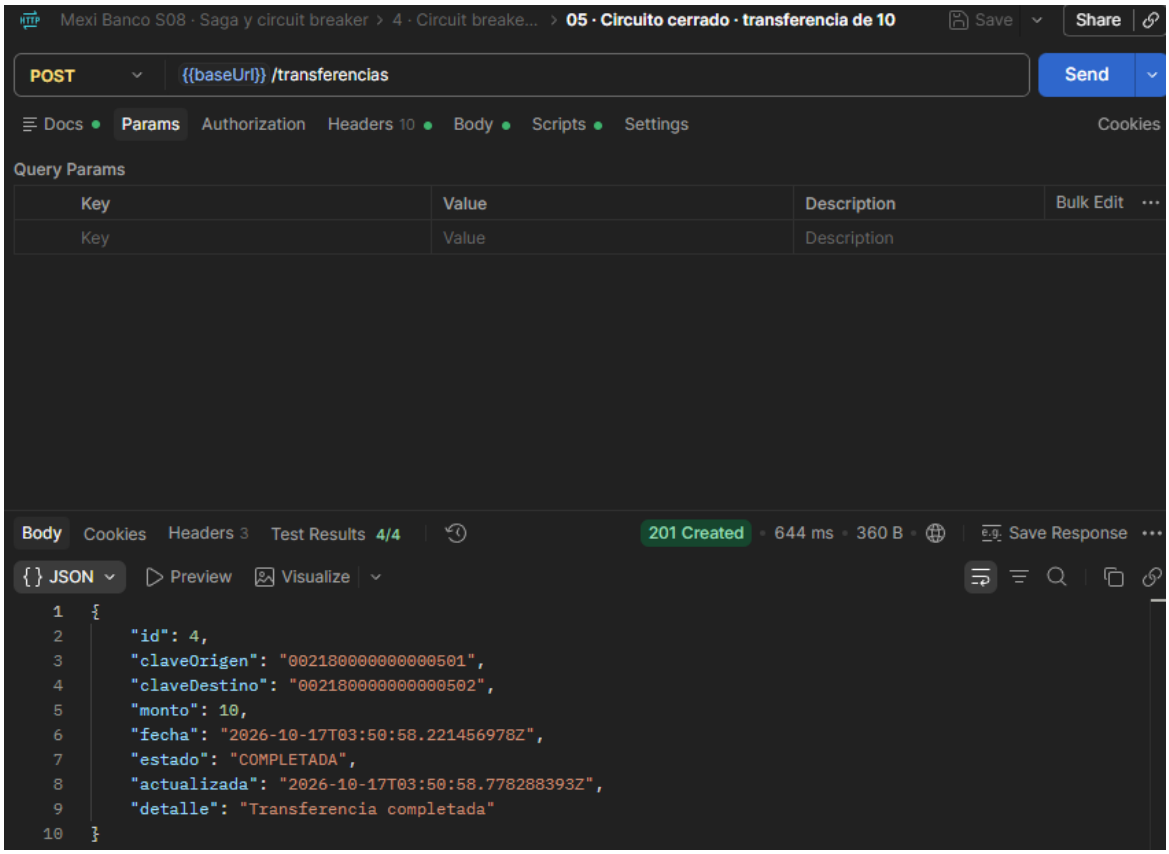


Figura 35: Transferencia de 10 completada con el circuito cerrado.

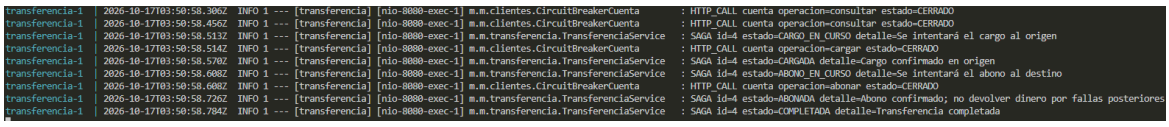


Figura 36: Logs del circuito cerrado y de la saga que terminó correctamente.

Después se apagó cuenta y se enviaron varias peticiones. Las fallas consecutivas incrementaron el contador hasta llegar al umbral de tres, y el circuito cambió de CERRADO a ABIERTO. En esta situación el servicio pudo responder 503 y guardar FALLIDA sin intentar el cargo, porque no se pudieron consultar las cuentas.

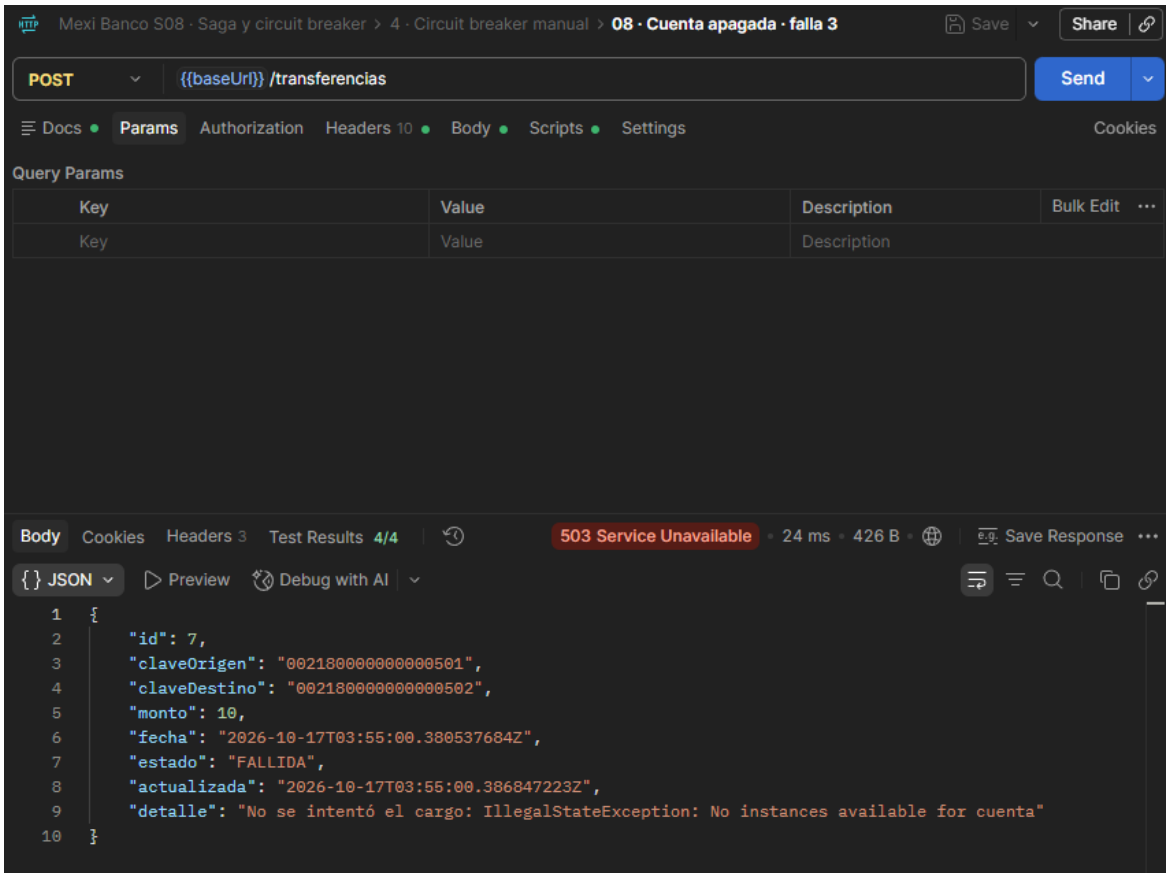


Figura 37: Petición con cuenta no disponible y respuesta 503 con estado FALLIDA.



Figura 38: Logs del contador de fallas y de la transición de CERRADO a ABIERTO.

Cuando se envió otra petición durante la apertura, el log mostró RECHAZO_SIN_HTTP. Esto comprueba que el circuito evitó otra llamada a cuenta. La respuesta rápida por sí sola no basta para probarlo, porque una ausencia de instancias en el balanceador también puede fallar rápido; aquí el mensaje del circuito indica el rechazo previo al HTTP.

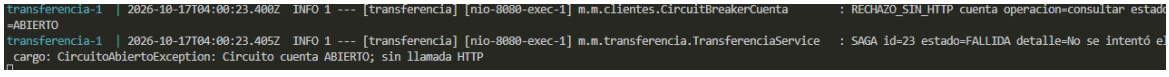


Figura 39: Rechazo de una nueva operación con el circuito abierto sin ejecutar HTTP.

Después de esperar al menos 15 segundos, una nueva petición permitió la prueba en SEMIABIERTO. Como cuenta seguía caída, esa prueba falló y el circuito regresó a

ABIERTO. La transición se hace al llegar la petición de prueba, no mediante un cambio de estado automático mientras no hay tráfico.

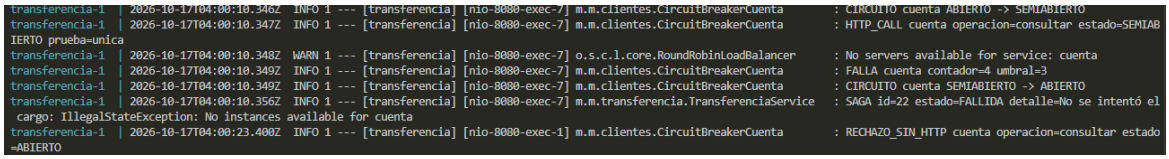


Figura 40: Transiciones ABIERTO a SEMIABIERTO y SEMIABIERTO a ABIERTO cuando la prueba vuelve a fallar.

Por último se recuperó cuenta y se envió otra petición de prueba. La transferencia terminó correctamente, lo que corresponde al cierre del circuito según la implementación. Se verificaron los saldos 980 y 520: hubo dos transferencias exitosas de 10, mientras las solicitudes rechazadas no movieron dinero.

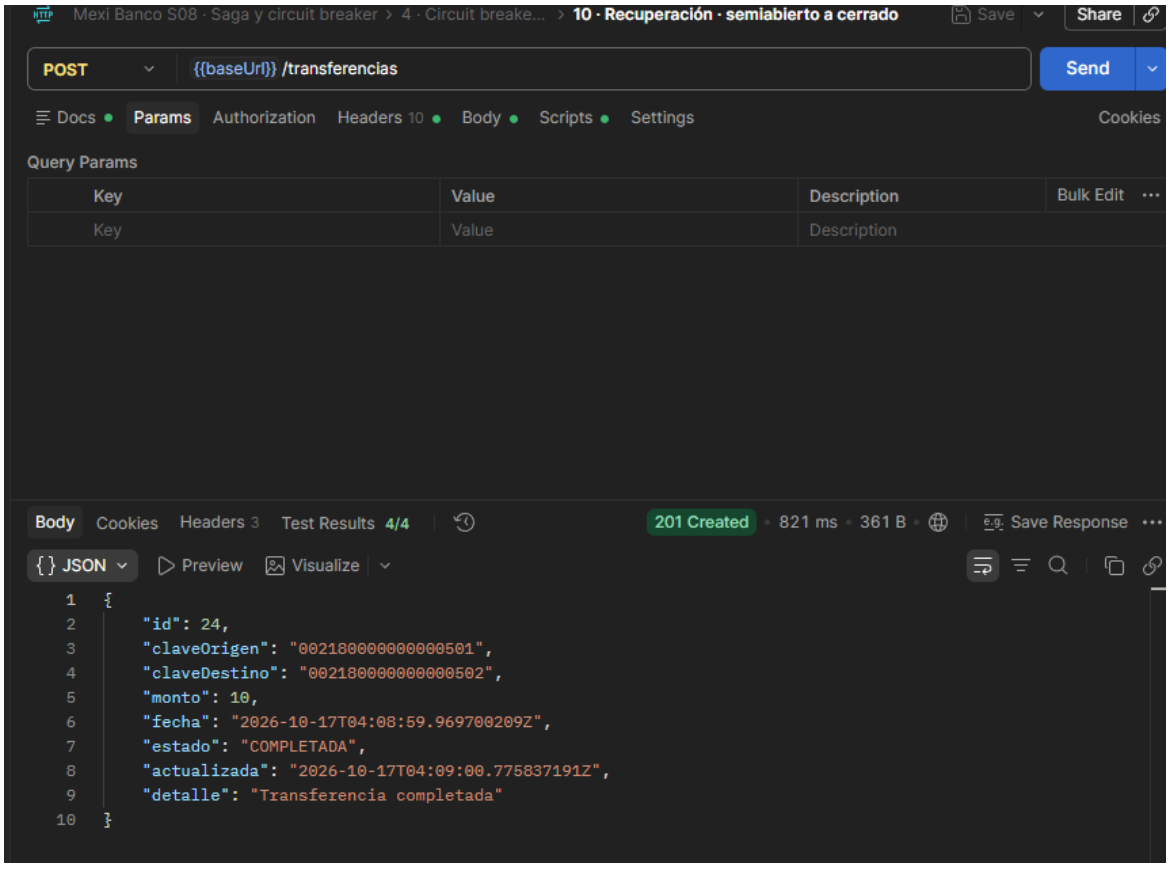


Figura 41: Petición de recuperación con respuesta 201 y transferencia COMPLETADA.

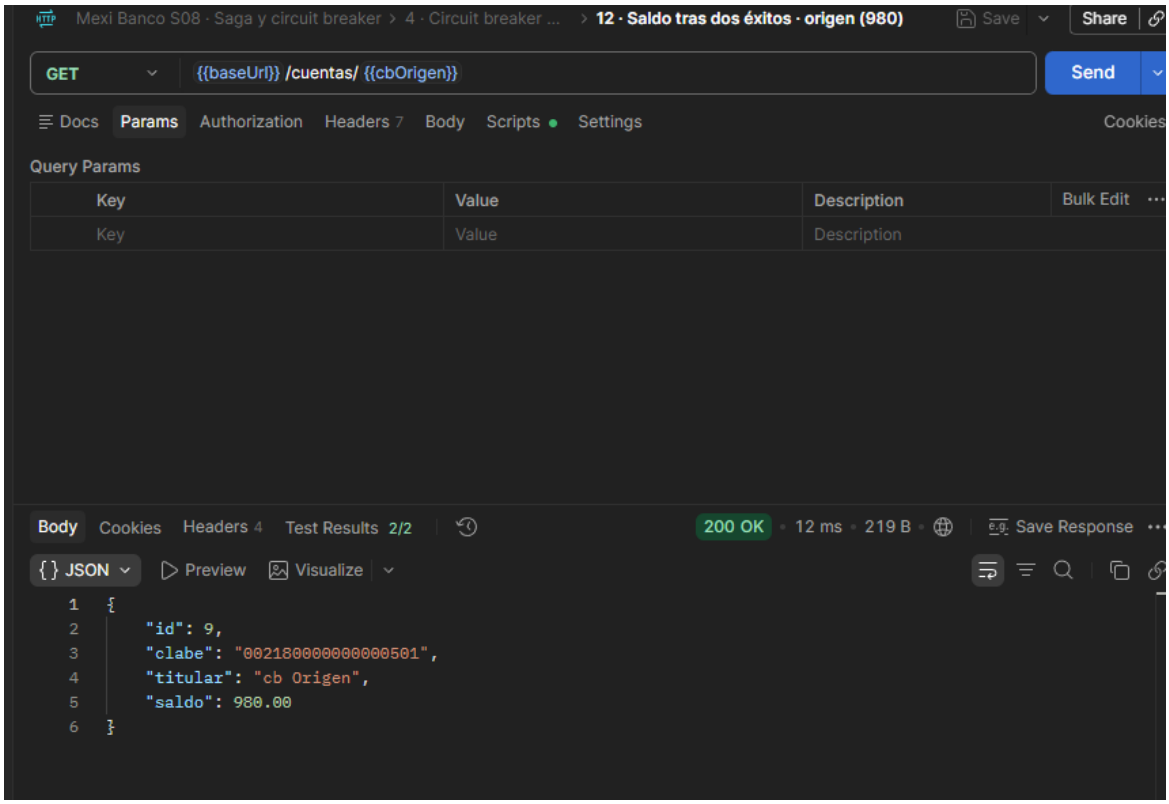


Figura 42: Saldo final del origen del escenario del circuito: 980.

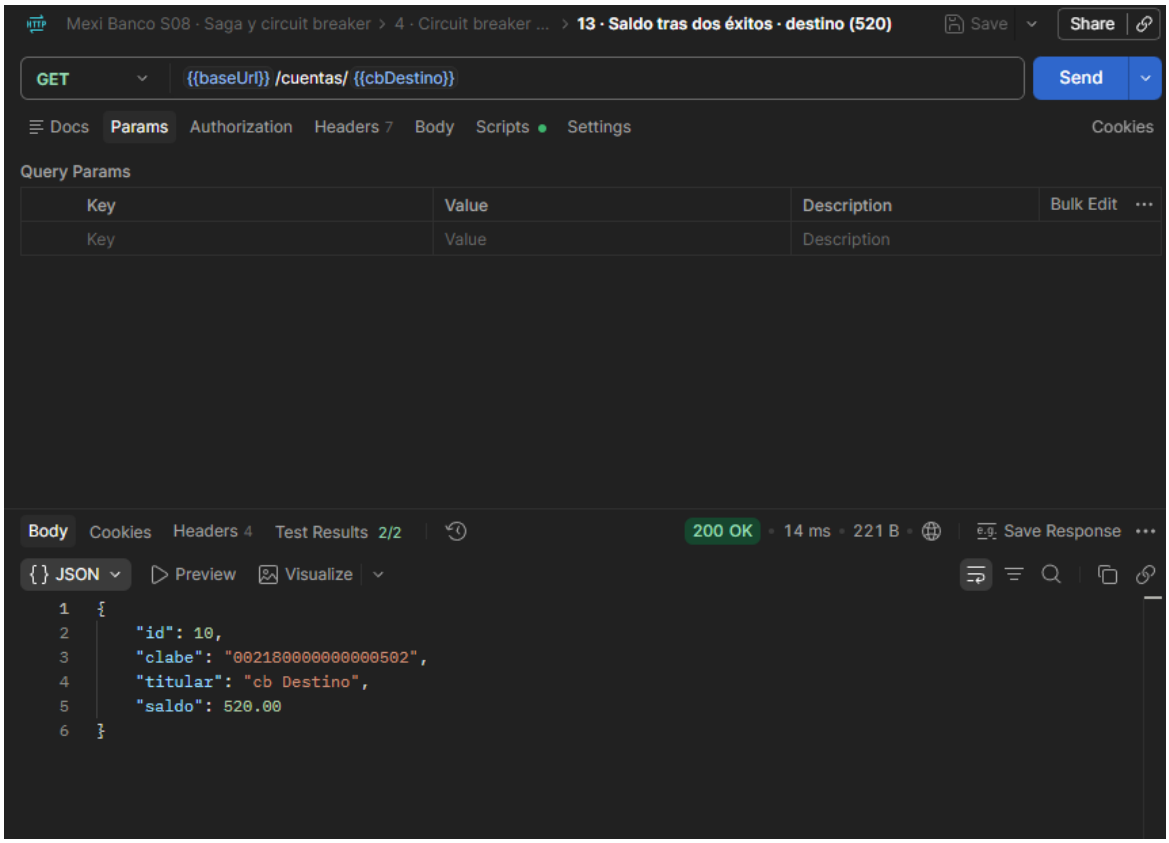


Figura 43: Saldo final del destino del escenario del circuito: 520.

8 Lo que la saga no proporciona

La saga no proporciona el aislamiento que se tenía dentro de una sola transacción del monolito. Entre el cargo y su compensación, alguien que consulte el origen puede ver 800 aunque después la devolución lo regrese a 1000. De hecho, en la pausa se pudo consultar ese saldo reducido por SQL. Otra transferencia podría encontrar fondos insuficientes durante ese intervalo. La compensación corrige el efecto económico después, pero no oculta los cambios intermedios ni garantiza que todas las operaciones distribuidas se ejecuten exactamente una vez.

Resultados y conclusión

La diferencia entre los escenarios se aprecia en los saldos y en el estado de la transferencia. Sin la saga, el origen quedó en 800 y el destino en 500 tras el error 500. Con una operación normal se obtuvieron 800 y 700, con estado COMPLETADA. Con una falla controlada y cuenta disponible, el origen volvió a 1000, el destino siguió en 500 y el estado quedó COMPENSADA. Cuando también falló la devolución, los saldos quedaron en 800 y 500, pero la transferencia conservó COMPENSACION_PENDIENTE.

El circuit breaker permitió observar cuándo se siguen intentando llamadas, cuándo se rechazan sin HTTP y cuándo se prueba la recuperación. Con esto se comprobó que compensar y proteger las llamadas resuelven problemas relacionados, pero distintos: la saga da seguimiento al dinero y el circuito limita las llamadas hacia un servicio que está fallando. Para completar la recuperación automática todavía hacen falta idempotencia remota, conciliación y reintentos de las compensaciones pendientes.